RKLLM3 Server 使用指南

文件标识：RK-YH-YF-436

发布版本：V1.2.0

日期：2026-10-23

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档详细介绍 RKLLM3 Server 的使用方法，包括服务启动、API 调用、Session/Slot 管理、KV Cache、模型扩展及多模态输入等功能。RKLLM3 Server 是基于 RKNPU3（Rockchip NPU）实现的 LLM HTTP Server，向客户端提供与 OpenAI API 兼容的 HTTP 接口。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V1.1.0	HPC	2026-08-06	初始版本	熊伟
V1.2.0	HPC	2026-10-23	1. 完善多卡级联推理功能 2. 支持LLM结构化输出（Grammar） 3. 支持设备侧模型加载 4. 支持加载多个lora权重文件 5. 新增 macOS arm64、Linux x86_64 平台支持	熊伟

目录

RKLLM3 Server 使用指南

1. RKLLM3 Server 概述

1.1 适用场景

1.2 系统架构

1.3 支持平台

2. 安装和启动

2.1 前置条件

2.1.1 环境要求

2.1.2 模型准备

2.2 模型和配置准备

2.2.1 模型目录结构

2.2.2 参数说明

2.2.3 配置文件说明

2.3 启动服务

2.3.1 命令行启动

2.3.2 配置文件启动

2.3.3 常用启动参数

2.3.4 加密模型加载

2.3.5 设备侧模型路径加载

2.4 健康检查

2.4.1 健康检查接口

2.4.2 服务状态验证

2.5 日志和启动问题

2.5.1 日志说明

2.5.2 常见启动问题

3. 快速调用

3.1 curl 调用

3.1.1 请求接口

3.1.2 示例

3.2 Python 客户端调用

3.2.1 SDK 使用

3.2.2 示例

3.3 流式和非流式请求

3.3.1 功能介绍

3.3.2 请求接口
3.3.3 示例
3.4 结果验证
3.4.1 响应格式
3.4.2 输出验证
4. Session（Slot）管理
4.1 多 Session
4.1.1 功能介绍
4.1.2 API
4.1.3 示例
4.1.4 使用限制
4.2 Slot 生命周期
4.3 暂停和继续
4.3.1 功能介绍
4.3.2 API
4.3.3 示例
4.4 打断
4.4.1 功能介绍
4.4.2 API
4.4.3 示例
4.5 并发调度机制
4.5.1 功能介绍
4.5.2 并发调度模型
4.5.3 Slot 分配规则
4.5.4 暂停/恢复/打断的响应
5. KV Cache
5.1 保存和加载
5.1.1 功能介绍
5.1.2 API
5.1.3 使用限制
5.1.4 示例
5.2 Checkpoint
5.2.1 功能介绍
5.2.2 API
5.2.3 示例

6. 模型扩展
6.1 LoRA
6.1.1 功能介绍
6.1.2 API
6.2 Tie Word Embedding
6.2.1 功能介绍
6.2.2 API
6.2.3 示例
6.3 Function Calling
6.3.1 功能介绍
6.3.2 API
6.3.3 示例
6.4 Embedding 模型
6.4.1 功能介绍
6.4.2 API
6.4.3 示例
7. 多模态输入
7.1 图像输入
7.1.1 功能介绍
7.1.2 API
7.1.3 示例
7.2 音频输入
7.2.1 功能介绍
7.2.2 API
7.2.3 示例
7.3 视频输入
7.3.1 功能介绍
7.3.2 API
7.3.3 示例
7.4 多模态请求格式
7.4.1 请求结构
7.4.2 Content 类型
7.4.3 示例
8. 多卡级联推理
8.1 功能介绍

8.2 配置参数
8.3 启动示例
8.4 KV Cache 保存与加载（多卡）
9. Grammar 语法约束
9.1 请求字段与优先级
9.2 EBNF 语法约束
9.2.1 功能介绍
9.2.2 API
9.2.3 示例
9.3 JSON Schema 结构化输出
9.3.1 功能介绍
9.3.2 API
9.3.3 示例
9.4 启用 grammar 功能与全局默认 grammar
9.4.1 功能介绍
9.4.2 API
9.4.3 示例
9.5 structural_tag 结构化约束
9.5.1 功能介绍
9.5.2 语法结构
9.5.3 示例
9.6 结构化输出与工具调用模式组合
9.6.1 功能介绍
9.6.2 组合规则
9.6.3 示例
9.7 结构化输出与思考模式组合
9.7.1 功能介绍
9.7.2 thinking_grammar 字段
9.7.3 示例
9.7.4 注意事项
附录
A. API 端点索引
B. OpenAI 兼容说明
B1. POST /v1/chat/completions：聊天补全
C. 错误码
C1. 错误类型速查表

C2. 常见错误消息及排查

D. 配置项汇总

D1. Server 配置

D2. 模型配置

1. RKLLM3 Server 概述

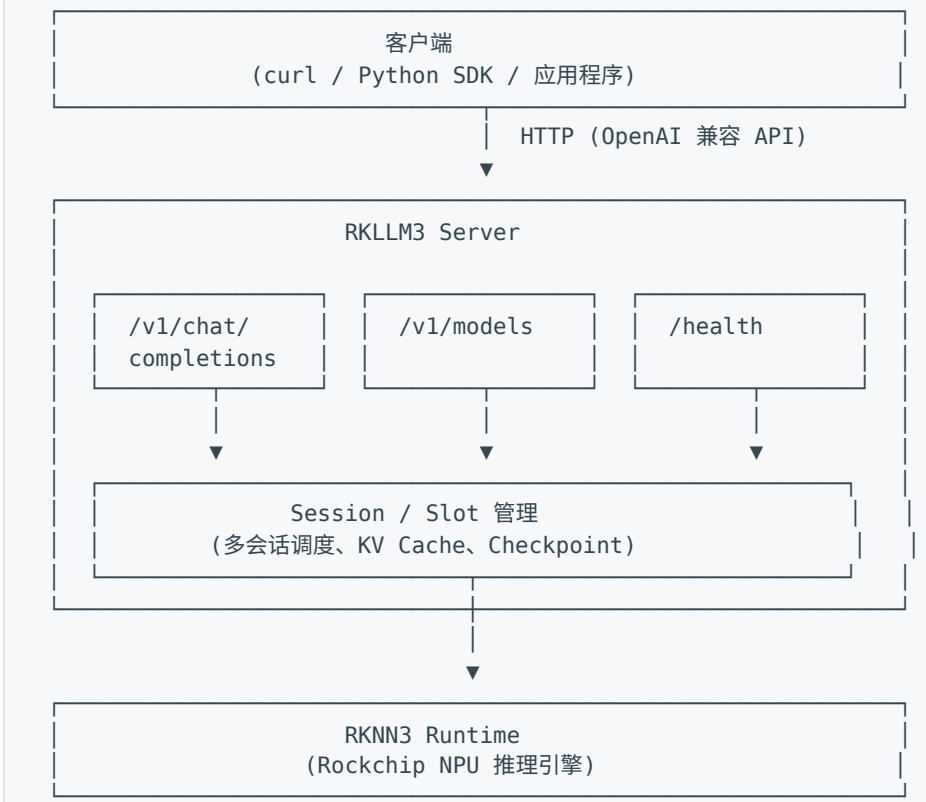
1.1 适用场景

RKLLM3 Server 是基于 RKNPU3（Rockchip NPU）实现的一套 LLM HTTP Server，向客户端提供与 OpenAI API 兼容的 HTTP 接口，适用于以下场景：

- 在 Rockchip 平台上部署大语言模型推理服务
- 为上层应用提供 OpenAI 兼容的对话补全、词嵌入等 API
- 支持多模态模型（图像、音频、视频理解）
- 多用户并发服务，通过 Session（Slot） 机制实现对话隔离

1.2 系统架构

RKLLM3 Server 的核心架构分为三层：



- **客户端**：通过 HTTP 协议发送 OpenAI 兼容的 API 请求
- **RKLLM3 Server**：负责接收和解析请求，管理 Session 调度，协调推理任务
- **RKNN3 Runtime**：底层 NPU 推理引擎，执行模型推理计算

1.3 支持平台

RKLLM3 Server 支持以下平台：

平台	架构
Linux	aarch64
Linux	x86_64
Android	arm64-v8a
Windows	x86_64
macOS	arm64

2. 安装和启动

2.1 前置条件

2.1.1 环境要求

RKLLM3 Server 的运行需要以下环境：

- Rockchip NPU 设备：RK3588/RK3576 开发板，或 Windows/macOS/Linux x86_64 PC + RK1820/1828 协处理器；RK3572
- RKNN3 Runtime 动态库
- 已转换的 RKNN 模型文件

2.1.2 模型准备

RKLLM3 Server 需要以下模型文件：

文件类型	说明	是否必需
LLM 模型（`.rknn`）	主语言模型文件，通过 `-m` 指定	必需
LLM 权重（`.weight`）	模型权重文件，通过 `--weight` 指定	必需
Vision 模型（`.rknn`）	多模态视觉模型，通过 `--model2` 指定	视觉时必需
Vision 权重（`.weight`）	视觉模型权重，通过 `--weight2` 指定	视觉时必需
Audio 模型（`.rknn`）	多模态音频模型，通过 `--model3` 指定	音频时必需
Audio 权重（`.weight`）	音频模型权重，通过 `--weight3` 指定	音频时必需
Tokenizer（`.gguf`）	词汇表文件，通过 `--vocab` 指定	必需
Embed（`.bin`）	Embedding 权重文件，通过 `--embed` 指定	非 tie-word 模式必需
Embed PLE（`.bin`）	Gemma4 模型的 per_layer_inputs embed 文件	Gemma4 模型必需
Rope Tensor（`.safetensors`）	Gemma4 模型的 rope cache 文件	Gemma4 模型必需
Mel Filter（`.txt`）	音频模型的 mel filters 文件，通过 `--mel-filter` 指定	部分音频模型必需

2.2 模型和配置准备

2.2.1 模型目录结构

建议将模型文件按以下结构组织：

```
models/
├── qwen2.5-3b/
│   ├── qwen2.5-3b.rknn
│   ├── qwen2.5-3b.weight
│   ├── qwen2.5-3b.tokenizer.gguf
│   └── qwen2.5-3b.embed.bin
├── qwen3-vl-2b/
│   ├── Qwen3-VL-2B-llm.rknn
│   ├── Qwen3-VL-2B-llm.weight
│   ├── Qwen3-VL-2B-vision.rknn
│   ├── Qwen3-VL-2B-vision.weight
│   ├── Qwen3-VL-2B-llm.tokenizer.gguf
│   └── Qwen3-VL-2B-llm.embed.bin
└── params.json
```

2.2.2 参数说明

RKLLM3 Server 支持通过命令行参数启动 HTTP 服务。

通用参数

参数	说明
`-a, --alias STRING`	设置模型别名（供 REST API 使用）
`-c, --ctx-size N`	提示词上下文大小（默认：4096，0 = 从模型加载）。支持 `#` 分隔符设置各 slot 独立值
`-n, --predict, --n-predict N`	要预测的 token 数量（默认：-1，-1 = 上下文大小）。支持 `#` 分隔符设置各 slot 独立值
`-m, --model FNAME`	LLM 模型路径，支持 `dev:` 前缀指定设备侧路径（详见 2.3.5 节）
`--weight FNAME`	LLM 权重路径（可不设置，未设置时由模型路径自动推导），支持 `dev:` 前缀指定设备侧路径
`--model2 FNAME`	多模态模型时的 vision 模型路径，支持 `dev:` 前缀指定设备侧路径
`--weight2 FNAME`	多模态模型时的 vision 权重路径（可不设置），支持 `dev:` 前缀指定设备侧路径
`--model3 FNAME`	多模态模型时的 audio 模型路径，支持 `dev:` 前缀指定设备侧路径
`--weight3 FNAME`	多模态模型时的 audio 权重路径（可不设置），支持 `dev:` 前缀指定设备侧路径
`--core-mask MASK`	LLM 模型执行的 NPU 核心掩码，支持十六进制或十进制。每一位控制一个 NPU 核心（例如 `0x01` = 仅 core 0，`0x03` = core 0+1，`0xff` = 全部核心）。默认：自动（使用全部可用核心）
`--core-mask2 MASK`	vision 模型（model2）执行的 NPU 核心掩码。默认：自动（使用全部可用核心）
`--core-mask3 MASK`	audio 模型（model3）执行的 NPU 核心掩码。默认：自动（使用全部可用核心）
`--vocab FNAME`	词汇表路径
`--embed FNAME`	embed 的 bin 文件路径
`--embed-ple FNAME`	Gemma4 模型的 per_layer_inputs embed 的 bin 文件路径
`--rope-tensor FNAME`	Gemma4 模型的 rope cache 的 safetensors 文件路径
`--mel-filter FNAME`	mel filters 路径（用于 audio 模型）

参数	说明
`--lora-weight FNAME`	LLM lora 权重路径。可重复传入以加载多个独立 `.lora_weight` 文件；每个文件可包含多个适配器，所有会话共享；可通过 `/lora-adapters` API 为各会话独立设置缩放比例
`--decrypt-key, --key FNAME`	RSA 加密的 LLM 模型用户密钥包路径（加载加密模型时必须配置）
`--decrypt-key2, --key2 FNAME`	RSA 加密的 Vision 模型用户密钥包路径（针对加密的多模态视觉模型）
`--decrypt-key3, --key3 FNAME`	RSA 加密的 Audio 模型用户密钥包路径（针对加密的多模态语音模型）
`--img-start STRING`	多模态模型的图像输入前缀
`--img-end STRING`	多模态模型的图像输入后缀
`--img-content STRING`	多模态模型的图像输入的占位符
`--audio-start STRING`	多模态模型的语音输入前缀
`--audio-end STRING`	多模态模型的语音输入后缀
`--audio-content STRING`	多模态模型的语音输入的占位符
`--video-start STRING`	qwen3_vl deepstack 视频模式的视频输入前缀
`--video-end STRING`	qwen3_vl deepstack 视频模式的视频输入后缀
`--video-content STRING`	qwen3_vl deepstack 视频模式的视频输入的占位符
`--img-width N`	多模态模型的输入图像的宽（部分裁剪过的模型需要设置，如 qwen3_vl）
`--img-height N`	多模态模型的输入图像的高（部分裁剪过的模型需要设置，如 qwen3_vl）
`--chat-template-file JINJA_TEMPLATE_FILE`	设置自定义 Jinja 聊天模板（默认：使用模型元数据中的模板）
`--embedding`	是否是词嵌入模型
`--reasoning-format FORMAT`	控制是否允许在响应中包含思考标签，以及如何返回该标签，格式可为： - `none`：思考内容保留在 `message.content` 中 - `deepseek`：将思考内容放入 `message.reasoning_content` （默认：`none`）
`--reasoning [on|off|auto]`	在对话中使用 reasoning/thinking（`on`、`off` 或 `auto`，默认：`auto`，从模板中获取）
`--record-path STRING`	设置保存用户请求的存放目录（默认为空，不保存）
`--n-session N`	session 个数，用于多 session 的场景（默认为 1，最大 16，需考虑设备内存）

采样参数

参数	说明
`--temp N`	温度（默认：0.8）

参数	说明
`--top-k N`	top-k 采样（默认：1，0 = 禁用）
`--top-p N`	top-p 采样（默认：0.9，1.0 = 禁用）
`--repeat-penalty N`	惩罚重复 token 序列（默认：1.0，1.0 = 禁用）
`--presence-penalty N`	重复 alpha 存在惩罚（默认：0.0，0.0 = 禁用）
`--frequency-penalty N`	重复 alpha 频率惩罚（默认：0.0，0.0 = 禁用）

专用参数

参数	说明
`-h, --help, --usage`	打印使用说明并退出
`--host HOST`	监听 IP 地址（默认：127.0.0.1）
`--port PORT`	监听端口（默认：8080）
`-to, --timeout N`	服务器读写超时时间（秒）（默认：600）
`--device-id STRING`	设备 ID（多设备时需要指定具体的设备 ID）
`--log-level N`	日志输出详细程度（默认：`0`）

2.2.3 配置文件说明

RKLLM3 Server 支持通过 JSON 配置文件（`params.json`）启动。使用 `--params-file` 参数指定配置文件时，将忽略命令行的其余参数，仅 `params.json` 内的参数有效。

配置文件的基本格式如下：

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "Qwen2.5-0.5B-Instruct.rknn",
            "weight": "Qwen2.5-0.5B-Instruct.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        }
    }
}
```


2.3 启动服务

2.3.1 命令行启动

以下示例展示了各场景下的命令行启动方式：

纯文本 LLM 模型（标准模式，需要 embed 文件）

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-
penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --
temp 0.8
```

LLM 模型（tie-word 模式，不需要 embed 文件）

```
./rkllm3-server -m Qwen3-VL-2B-llm-tie-word.rknn --weight Qwen3-VL-2B-llm-tie-
word.weight --vocab Qwen3-VL-2B-llm_quant.tokenizer.gguf --tie-word-embedding 1 --
host 0.0.0.0 --port 8080 -c 4096 --n-predict 512 --repeat-penalty 1.1 --presence-
penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8
```

Gemma4 模型

```
./rkllm3-server -m gemma-4.rknn --vocab gemma-4.tokenizer.gguf --embed gemma-
4.embed.bin --embed-ple gemma-4_per_layer_inputs.embed.bin --rope-tensor gemma-
4.safetensors --host 0.0.0.0 --port 8080 -c -1 --n-predict 2048 --repeat-penalty 1.0
--presence-penalty 0 --frequency-penalty 0 --top-k 1 --top-p 0.8 --temp 0.8
```

多模态模型（LLM + Vision）

```
# MiniCPM-3o
./rkllm3-server -m MiniCPM-3o-llm.rknn --model2 MiniCPM-3o-vision.rknn --vocab
MiniCPM-3o-llm.tokenizer.gguf --embed MiniCPM-3o-llm.embed.bin --host 0.0.0.0 --port
8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-
penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<image>" --img-end "
</image>" --img-content "<unk>"

# Qwen2.5-VL-3B
./rkllm3-server -m Qwen2.5-VL-3B-llm.rknn --model2 Qwen2.5-VL-3B-vision.rknn --vocab
Qwen2.5-VL-3B-llm.tokenizer.gguf --embed Qwen2.5-VL-3B-llm.embed.bin --host 0.0.0.0
--port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 392 --img-height 392

# Qwen3-VL-2B
./rkllm3-server -m Qwen3-VL-2B-llm.rknn --model2 Qwen3-VL-2B-vision.rknn --vocab
Qwen3-VL-2B-llm.tokenizer.gguf --embed Qwen3-VL-2B-llm.embed.bin --host 0.0.0.0 --
port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 384 --img-height 384

# Qwen3.5-4B
./rkllm3-server -m Qwen3.5-4B.rknn --model2 Qwen3.5-4B-vision.rknn --vocab Qwen3.5-
4B.tokenizer.gguf --embed Qwen3.5-4B.embed.bin --host 0.0.0.0 --port 8080 -c 0 --n-
predict 2048 --repeat-penalty 1.0 --presence-penalty 1.5 --frequency-penalty 0 --
top-k 1 --top-p 0.9 --temp 1.0 --img-start "<|vision_start|>" --img-end "
<|vision_end|>" --img-content "<|image_pad|>" --img-width 384 --img-height 384
```

多模态模型（LLM + Audio）

```
./rkllm3-server -m gemma-4.rknn --model3 gemma-4-audio.rknn --vocab gemma-
4.tokenizer.gguf --embed gemma-4.embed.bin --embed-ple gemma-
4_per_layer_inputs.embed.bin --rope-tensor gemma-4.safetensors --host 0.0.0.0 --port
8080 -c -1 --n-predict 2048 --repeat-penalty 1.0 --presence-penalty 0 --frequency-
penalty 0 --top-k 1 --top-p 0.8 --temp 0.8 --audio-start "<|audio>" --audio-end "
<audio|>" --audio-content "<|audio|>"
```

多模态模型（LLM + Vision + Audio）

```
./rkllm3-server -m Qwen2.5-Omni-3B-llm.rknn --model2 Qwen2.5-Omni-3B-vision.rknn --
model3 Qwen2.5-Omni-3B-audio.rknn --vocab Qwen2.5-Omni-3B-llm.tokenizer.gguf --embed
Qwen2.5-Omni-3B-llm.embed.bin --mel-filter mel_128_filters.txt --host 0.0.0.0 --port
8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-
penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<|vision_bos|>" --img-end
"<|vision_eos|>" --img-content "<|IMAGE|>" --audio-start "<|audio_bos|>" --audio-end
"<|audio_eos|>" --audio-content "<|AUDIO|>"
```

词嵌入模型

```
./rkllm3-server -m Qwen3-Embedding-4B.rknn --vocab Qwen3-Embedding-4B.tokenizer.gguf
--embed Qwen3-Embedding-4B.embed.bin --embedding
```

2.3.2 配置文件启动

同时加载多个模型时，通过 `--params-file` 指定 JSON 配置文件：

```
./rkllm3-server --params-file params.json
```

多模型配置文件示例：

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "Qwen2.5-0.5B-Instruct.rknn",
            "weight": "Qwen2.5-0.5B-Instruct.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        },
        "Qwen3-0.6B": {
            "alias": "Qwen3-0.6B",
            "model": "Qwen3-0.6B.rknn",
            "weight": "Qwen3-0.6B.weight",
            "model2": "",
            "weight2": "",
            "model3": "",
            "weight3": "",
            "core-mask": 0,
            "core-mask2": 0,
            "core-mask3": 0,
```

```
            "vocab": "Qwen3-0.6B.tokenizer.gguf",
            "embed": "Qwen3-0.6B.embed.bin",
            "mel-filter": "",
            "lora-weight": "",
            "decrypt-key": "",
            "decrypt-key2": "",
            "decrypt-key3": "",
            "ctx-size": 1024,
            "predict": 512,
            "temp": 0.8,
            "top-k": 1,
            "top-p": 0.8,
            "repeat-penalty": 1.1,
            "presence-penalty": 1.0,
            "frequency-penalty": 1.0,
            "img-start": "",
            "img-end": "",
            "img-content": "",
            "audio-start": "",
            "audio-end": "",
            "audio-content": "",
            "video-start": "",
            "video-end": "",
            "video-content": "",
            "img-width": 0,
            "img-height": 0,
            "chat-template-file": "",
            "embedding": false,
            "record-path": "",
            "reasoning-format": "none",
            "reasoning": "auto",
            "n-session": 1
        }
    }
}
```

2.3.3 常用启动参数

以下为最常用的启动参数组合：

场景	最小参数集
纯文本 LLM	`-m <model> --vocab <vocab> --embed <embed>`
Tie-word LLM	`-m <model> --vocab <vocab> --tie-word-embedding 1`
LLM + Vision	上述加上 `--model2 <vision>` `--img-start` `--img-end` `--img-content`
LLM + Audio	上述加上 `--model3 <audio>` `--audio-start` `--audio-end` `--audio-content`
Embedding 模型	`-m <model> --vocab <vocab> --embed <embed> --embedding`
加密模型	上述加上 `--decrypt-key`（以及 `--decrypt-key2` / `--decrypt-key3` 按需）
设备侧模型	`-m <model> --vocab <vocab> --embed <embed>`，模型路径添加 `dev:` 前缀（详见 2.3.5 节）

2.3.4 加密模型加载

RKLLM3 Server 支持加载 RSA 加密的模型文件。加密模型的文件扩展名通常为 `.rknn.enc`，需要配合对应的用户密钥包文件（`.key`）使用。

参数说明

加密模型加载通过以下参数配置密钥包路径：

参数	对应模型	说明
`--decrypt-key, --key`	LLM 模型（model）	必须与加密的 LLM 模型文件配合使用
`--decrypt-key2, --key2`	Vision 模型（model2）	仅当 Vision 模型也加密时需配置
`--decrypt-key3, --key3`	Audio 模型（model3）	仅当 Audio 模型也加密时需配置

命令行示例

```
# 加载加密模型（LLM 纯文本）
./rkllm3-server -m Qwen3-4B.rknn.enc --decrypt-key Qwen3-4B.key --vocab Qwen3-
4B.tokenizer.gguf --embed Qwen3-4B.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-
predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --
top-k 1 --top-p 0.8 --temp 0.8

# 加载加密模型（LLM+VISION 多模态）
./rkllm3-server -m Qwen3-VL-4B-llm.rknn.enc --model2 Qwen3-VL-4B-vision.rknn.enc --
decrypt-key Qwen3-VL-4B-llm.key --decrypt-key2 Qwen3-VL-4B-vision.key --vocab Qwen3-
VL-4B-llm.tokenizer.gguf --embed Qwen3-VL-4B-llm.embed.bin --host 0.0.0.0 --port
8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --frequency-
penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "<|vision_start|>" --img-
end "<|vision_end|>" --img-content "<|image_pad|>" --img-width 384 --img-height 384
--video-start "<|vision_start|>" --video-end "<|vision_end|>" --video-content "
<|video_pad|>"

# 加载加密模型（LLM+VISION+AUDIO 全模态）
./rkllm3-server -m Qwen3-Omni-4B-llm.rknn.enc --model2 Qwen3-Omni-4B-vision.rknn.enc
--model3 Qwen3-Omni-4B-audio.rknn.enc --decrypt-key Qwen3-Omni-4B-llm.key --decrypt-
key2 Qwen3-Omni-4B-vision.key --decrypt-key3 Qwen3-Omni-4B-audio.key --vocab Qwen3-
Omni-4B-llm.tokenizer.gguf --embed Qwen3-Omni-4B-llm.embed.bin --mel-filter
mel_128_filters.txt --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --repeat-
penalty 1.1 --presence-penalty 1.0 --frequency-penalty 1.0 --top-k 1 --top-p 0.8 --
temp 0.8 --img-start "<|vision_start|>" --img-end "<|vision_end|>" --img-content "
<|image_pad|>" --audio-start "<|audio_start|>" --audio-end "<|audio_end|>" --audio-
content "<|audio_pad|>"
```

配置文件方式

通过 `params.json` 加载加密模型时，在对应模型的配置中添加 `decrypt-key`、`decrypt-key2`、`decrypt-key3` 字段即可：

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen3-4B-encrypted": {
            "alias": "Qwen3-4B-encrypted",
            "model": "Qwen3-4B.rknn.enc",
            "decrypt-key": "Qwen3-4B.key",
            "vocab": "Qwen3-4B.tokenizer.gguf",
            "embed": "Qwen3-4B.embed.bin",
            "ctx-size": 768,
            "predict": 512
        }
    }
}
```

2.3.5 设备侧模型路径加载

RKLLM3 Server 支持加载位于设备侧文件系统（如 eMMC）中的模型文件。在模型或权重路径前添加 `dev:` 前缀即可，例如 `dev:/data/llm.rknn`。使用 `dev:` 前缀时，服务将调用设备侧加载接口（`rknn3_load_model_from_device`），并跳过主机侧的文件存在性检查。

命令行示例

```
# LLM 模型与权重位于设备侧文件系统
./rkllm3-server -m dev:/data/qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --
embed qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --top-k
1 --top-p 0.8 --temp 0.8
```

配置文件示例

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "timeout": 30,
    "models": {
        "Qwen2.5-0.5B": {
            "alias": "Qwen2.5-0.5B",
            "model": "dev:/data/Qwen2.5-0.5B-Instruct.rknn",
            "weight": "dev:/data/Qwen2.5-0.5B-Instruct.weight",
            "vocab": "Qwen2.5-0.5B-Instruct.tokenizer.gguf",
            "embed": "Qwen2.5-0.5B-Instruct.embed.bin",
            "ctx-size": 1024,
            "predict": 512
        }
    }
}
```

使用说明

- **权重路径自动推导**：`--weight`、`--weight2`、`--weight3` 未设置时，权重路径由模型路径将 `.rknn` 替换为 `.weight` 自动推导，`dev:` 前缀会一并保留。例如 `-m dev:/data/llm.rknn` 自动推导权重为 `dev:/data/llm.weight`。

- **多卡级联推理**：仅需指定第一段（`_seg0`）路径，其余分段路径自动推导，`dev:` 前缀同样会一并保留。详见 8. 多卡级联推理。
- **同组路径必须同侧**：同一模型的模型路径与权重路径必须同时位于设备侧或主机侧。模型路径带 `dev:` 前缀而权重路径不带（或反之）时不允许混用，将输出错误日志：`model path has "dev:" prefix but weight "<路径>" does not, mixing device/host paths is not supported!`（vision/audio 模型对应 `model2` / `model3`）。
- **不同模型可分别指定**：多模态场景下各模型可独立选择加载侧。例如 LLM 模型使用 `dev:` 前缀（设备侧），Vision/Audio 模型使用主机侧路径。
- **加密模型**：设备侧加密模型同样支持加载，用户密钥包（`.key`）始终从主机侧文件系统读取，`--decrypt-key`、`--key2`、`--key3` 不受 `dev:` 前缀影响。
- **日志显示**：加载时日志中的模型路径包含 `dev:` 前缀（如 `loading model 'dev:/data/llm.rknn'`），属正常现象。

> **注意**：`dev:` 前缀仅适用于模型与权重路径（`-m/--model`、`--weight`、`--model2`、`--weight2`、`--model3`、`--weight3`），不适用于 `--vocab`、`--embed`、`--rope-tensor`、`--mel-filter` 等其他文件路径参数，这些文件仍需位于主机侧文件系统。

2.4 健康检查

2.4.1 健康检查接口

RKLLM3 Server 提供 `/health` 端点用于检查服务状态。

请求

```
GET /health
```

返回结果

模型正在加载时（HTTP 503）：

```
{
    "error": {
        "code": 503,
        "message": "Loading model",
        "type": "unavailable_error"
    }
}
```

模型加载完成且服务就绪时（HTTP 200）：

```
{
    "status": "ok"
}
```

2.4.2 服务状态验证

启动服务后，建议通过以下命令验证服务是否就绪：

```
curl http://localhost:8080/health
```

如果返回 `{"status": "ok"}`，说明服务已正常启动。

2.5 日志和启动问题

2.5.1 日志说明

RKLLM3 Server 通过 `--log-level N` 参数控制日志输出等级（默认：`0`），控制输出格式的详细程度：

值	日志截断	请求体打印	Prompt Token 日志
`0`（默认）	截断到 1024 字节	仅非多媒体请求（不含 image/audio）	截断到 1024 个
`1`	不截断，完整输出	仅非多媒体请求（不含 image/audio）	截断到 1024 个
`≥2`	不截断，完整输出	全部请求（含多媒体 base64）	全部打印

2.5.2 常见启动问题

参数配置错误

问题描述	对应日志
命令行参数不存在或拼写错误	`error: invalid argument: <错误参数>`
参数缺少值（如 `-m` 后未指定模型路径）	`expected value for argument`
`--core-mask` 值格式错误	`invalid value for --core-mask: "<值>"`
`--params-file` 指定的配置文件不存在	`Can not open the params-file "<路径>"!`
`params.json` 中 `models` 字典为空	`The "models" dict in params-file must have at least one subitem!`
`reasoning-format` 值无效（仅支持 `none`、`deepseek`）	`The "reasoning-format" is invalid!`
`reasoning` 值无效（仅支持 `on`、`off`、`auto`）	`The "reasoning" is invalid!`
`--chat-template-file` 指定的模板文件不存在	`failed to open file "<路径>"`
提供的 chat template 格式不支持	`error: the supplied chat template is not supported`
同时使用 `--embedding` 和 `--reranking`	`error: either --embedding or --reranking can be specified, but not both`

模型文件相关

问题描述	对应日志
未指定模型路径（`-m` 参数缺失）	`model path is empty`
模型文件（`.rknn`）不存在	`the model file "<路径>" does not exist!`
权重文件（`.weight`）不存在	`the model file "<路径>" does not exist!`

问题描述	对应日志
设备侧模型路径与主机侧权重路径混用（`dev:` 前缀不一致）	`model path has "dev:" prefix but weight "<路径>" does not, mixing device/host paths is not supported!`
设备侧模型加载失败（`dev:` 前缀路径）	`rknn3_load_model_from_device failed! ret=<错误码>`
模型加载失败（文件损坏或格式不兼容）	`rknn3_load_model_from_path failed! ret=<错误码>`
模型初始化失败	`rknn3_model_init failed! ret=<错误码>`
查询模型配置失败	`rknn3_query RKNN3_QUERY_LLM_CONFIG failed! ret=<错误码>`
模型加载综合失败	`failed to load model, '<路径>'`
Embed 文件（`.bin`）不存在	`Failed to open embed file: <路径>`
Embed 文件 mmap 失败	`Failed to mmap file`
Embed-ple 文件不存在（Gemma4 模型）	`Failed to open embed-ple file: <路径>`
词汇表文件（`.gguf`）不存在或格式错误	`cannot find tokenizer vocab in model file`
词汇表 tokenizer 类型未知	`unknown tokenizer: '<类型>'`

设备与运行时

问题描述	对应日志
未找到 NPU 设备	`no device found`
NPU 设备检测失败	`rknn3_find_devices fail! ret=<错误码>`
指定的 `--device-id` 无效	`the device-id "<id>" is invalid! please check it carefully!`
RKNN3 Runtime 初始化失败	`rknn3_init fail! ret=<错误码>`
RKNN3 Session 初始化失败	`Failed to initialize session!`
RKNN3 查询失败	`rknn_query fail! ret=<错误码>`
端口被占用（`--port`）	`couldn't bind HTTP server socket, hostname: <host>, port: <port>`
HTTP Server 初始化失败	`Failed to initialize http server`
HTTP Server 启动失败	`Failed to start http server`
模型加载阶段退出	`exiting due to model loading error`

多模态相关

问题描述	对应日志
Vision 模型（model2）输入尺寸未知	`the input image size of model2 is unknow, please set by '--img-width' & '--img-height'!`
Vision 模型类型不支持	`model2 (<模型名称>) is not supported!`

其他

问题描述	对应日志
请求中 `tie_word_embedding=1` 但模型未启用该模式	`WRN: tie_word_embedding=1 requested but model was not initialized with --tie-word-embedding`
`tie_word_embedding` 模式未加载 embed 数据	`tie_word_embedding requires embedding file, but no embedding data was loaded`
KV Cache 满（上下文不够）	`failed to decode the batch: KV cache is full - try increasing it via the context size`

3. 快速调用

3.1 curl 调用

3.1.1 请求接口

RKLLM3 Server 提供 OpenAI 兼容的 `/v1/chat/completions` 端点用于对话补全。

请求

```
POST /v1/chat/completions
```

3.1.2 示例

使用 curl 发送对话请求：

```
curl --request POST --url http://localhost:8080/v1/chat/completions \
    --header "Content-Type: application/json" \
    --data '{"messages": [{"role": "user", "content": "Hello!"}],"n_predict": 128}'
```

3.2 Python 客户端调用

3.2.1 SDK 使用

RKLLM3 Server 兼容 OpenAI Python SDK，可直接使用 `openai` 库进行调用。

3.2.2 示例

纯文本对话

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "system", "content": "You are ChatGPT, an AI assistant. Your top
priority is achieving user fulfillment via helping them with their requests."},
        {"role": "user", "content": "Write a limerick about python exceptions"}
    ]
)

print(completion.choices[0].message)
```

多模态对话（图像 + 音频）

```python
import base64
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8080/v1",
    api_key="sk-no-key-required"
)

def encode_base64(media_path):
    with open(media_path, "rb") as media_file:
        return base64.b64encode(media_file.read()).decode("utf-8")

image_path = "demo.jpg"
audio_path = "demo.wav"     # 如存在音频输入

base64_image = encode_base64(image_path)
base64_audio = encode_base64(audio_path)    # 如存在音频输入

completion = client.chat.completions.create(
    model="Qwen2.5-VL",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "image_url",
                    "image_url": {
                        "url": f"{base64_image}",
                    },
                },
                {
                    "type": "input_audio",
                    "input_audio": {
                        "data": f"{base64_audio}",
                        "format": "wav",
                    }
                },  # 如存在音频输入
                {"type": "text", "text": "请描述一下图片?"},
            ],
        }
    ],
    stream=True,
    extra_body={
        "n_predict": 256,
        "chat_template_kwargs": {"enable_thinking": False},     # Thinking 类模型（如Qwen3）可以通过此方式关闭 thinking 输出
        "id_slot": 0,           # 当 --n-session 参数大于 1 时，可以通过此参数来切换不同的session
        # 采样参数
        "top_k": 1,
        "top_p": 0.9,
        "temperature": 0.8,
        "repeat_penalty": 1.1,
        "frequency_penalty": 1.0,
        "presence_penalty": 1.0,
    }
)

for chunk in completion:
    delta = chunk.choices[0].delta
```

```
    if delta.content:
        print(delta.content, end='', flush=True)
    if chunk.choices[0].finish_reason == "stop":
        break
print('')
```

3.3 流式和非流式请求

3.3.1 功能介绍

RKLLM3 Server 支持两种响应模式：

- **非流式**（`stream=False`）：等待完整生成后一次性返回结果
- **流式**（`stream=True`）：逐 token 返回生成结果，基于 SSE（Server-Sent Events）协议

3.3.2 请求接口

请求

```
POST /v1/chat/completions
```

通过 `stream` 参数控制模式。

3.3.3 示例

非流式请求（curl）

```
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [
            {
                "role": "system",
                "content": "You are ChatGPT, an AI assistant. Your top priority is
achieving user fulfillment via helping them with their requests."
            },
            {
                "role": "user",
                "content": "Write a limerick about python exceptions"
            }
        ]
    }'
```

流式请求（Python）

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "Hello!"}
    ],
    stream=True
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
```

3.4 结果验证

3.4.1 响应格式

RKLLM3 Server 的响应格式与 OpenAI API 兼容。非流式响应的基本结构如下：

```
{
    "id": "chatcmpl-xxx",
    "object": "chat.completion",
    "created": 1735142223,
    "model": "Qwen2.5-3B",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "..."
            },
            "finish_reason": "stop"
        }
    ]
}
```

3.4.2 输出验证

`finish_reason` 字段表示结束原因：

值	含义
`stop`	正常结束
`length`	达到 `max_tokens` / `n_predict` 上限被截断
`tool_calls`	触发了工具调用

4. Session（Slot）管理

> **说明**：本文档中的「session」和「slot」指代同一概念：一个独立的推理槽位，拥有独立的 KV Cache 和对话上下文。CLI 参数 `--n-session` 和 JSON 配置字段 `n-session` 控制 Slot 的数量；API 端点使用 `/slots/{slot_id}` 路由，请求参数使用 `id_slot`。两者可互换理解。

4.1 多 Session

4.1.1 功能介绍

`--n-session` 参数用于创建多个独立的会话（slot），每个会话拥有独立维护的 KV Cache 和对话上下文，支持处理多个用户的对话请求。

当 `--n-session` 大于 1 时，服务端会创建对应数量的 slot，客户端在请求中通过 `id_slot` 参数指定使用哪个 slot（id 从 0 开始，取值范围 `[0, --n-session - 1]`，默认为 0）。

使用场景

- **多用户并发服务**：多个用户同时对话，各自独立的 KV Cache，互不干扰
- **对话隔离**：不同任务/话题使用不同 slot，避免上下文混淆

4.1.2 API

请求

```
POST /v1/chat/completions
```

通过 `id_slot` 参数指定使用的 slot。

参数	类型	必填	描述
`id_slot`	int	否	指定当前 slot 的 id（当 `--n-session` 参数大于 1 时，可以通过此参数来切换不同的 session，该值需小于 `--n-session`，默认为 0）

4.1.3 示例

启动服务（创建 3 个 session）

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080 -c 768 --n-predict 512 --n-session 3
```

curl 示例

```
# Slot 0 对话
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [{"role": "user", "content": "你好，我叫小明"}],
        "id_slot": 0
    }'

# Slot 1 对话（独立上下文，不知道 Slot 0 的对话内容）
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer no-key" \
    -d '{
        "messages": [{"role": "user", "content": "你好，我叫小红"}],
        "id_slot": 1
    }'
```

4.1.4 使用限制

各 Slot 独立上下文大小

当使用 `--n-session N`（多 session 模式）时，每个 slot 可以拥有不同的上下文大小（ctx-size），满足不同场景对内存和上下文长度的差异化需求。可通过 CLI 的 `#` 分隔符或 `params.json` 的 JSON 数组来分别设置。

CLI 示例（3 个 slot，各自不同的 ctx-size）：

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin \
    --host 0.0.0.0 --port 8080 --n-session 3 --n-predict 512 \
    -c 4096#2048#1024
```

params.json 示例（相同配置，使用 JSON 数组）：

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./qwen2.5-3b.rknn",
        "vocab": "./qwen2.5-3b.tokenizer.gguf",
        "embed": "./qwen2.5-3b.embed.bin",
        "ctx-size": [4096, 2048, 1024],
        "predict": 512,
        "n-session": 3
    }]
}
```

`"n-session": 3` 时，各 slot 的分配如下：

Slot	ctx-size	适用场景
0	4096	长文档/多轮对话
1	2048	常规对话

Slot	ctx-size	适用场景
2	1024	短问答/资源受限

如果数组元素个数少于 `n-session`，剩余 slot 将使用标量默认值。也支持标量值（如 `"ctx-size": 4096`），向后兼容地为所有 slot 设置相同的值。

> **注意**：各 slot 的 ctx-size 会影响 NPU 内存分配，较大的 ctx-size 会消耗更多内存。请根据实际内存情况合理配置。

各 Slot 独立预测长度

多 session 模式下，每个 slot 也可以拥有不同的预测 token 数（n-predict），满足不同场景对生成长度的差异化需求。同样通过 CLI 的 `#` 分隔符或 `params.json` 的 JSON 数组来分别设置。

CLI 示例（3 个 slot，各自不同的 n-predict）：

```
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin \
    --host 0.0.0.0 --port 8080 --n-session 3 -c 4096 \
    -n 512#256#-1
```

params.json 示例（相同配置，使用 JSON 数组）：

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./qwen2.5-3b.rknn",
        "vocab": "./qwen2.5-3b.tokenizer.gguf",
        "embed": "./qwen2.5-3b.embed.bin",
        "ctx-size": 4096,
        "predict": [512, 256, -1],
        "n-session": 3
    }]
}
```

`"n-session": 3` 时，各 slot 的分配如下：

Slot	n-predict	含义
0	512	最多生成 512 个 token
1	256	最多生成 256 个 token
2	-1	设置为上下文的长度

> **注意**：每个请求还可以通过 `extra_body` 中的 `n_predict` 字段进一步覆盖当前 slot 的默认值，提供更灵活的控制。

4.2 Slot 生命周期

每个 slot 在推理过程中会经历不同的生命周期状态。RKLLM3 Server 提供了 slot的暂停、恢复和打断等操作接口，支持对推理任务进行精细化调度。

slot 共有 5 种状态：

状态	说明
IDLE	Slot 初始状态，无任务正在处理，可接受新的推理请求
STARTED	任务已分配到 Slot，正在初始化任务
RUNNING	正在运行，调用 RKNN3 Runtime 执行推理
PAUSED	Slot 被暂停，KV Cache 保留在内存中，可通过 Resume 恢复到暂停前的状态
ERROR	发生致命错误，Slot 不可用，需人工干预或重启服务

4.3 暂停和继续

4.3.1 功能介绍

Pause/Resume 允许控制指定 slot 的推理状态。Pause 在不销毁 KV Cache 的前提下暂停推理，Resume 从暂停位置恢复。适用于资源调度、多 slot 间动态分配推理时间片等场景。

4.3.2 API

POST `/slots/{slot_id}/pause`：暂停 slot

暂停指定 slot 的推理，slot 的 KV Cache 将被保留。

请求

```
POST /slots/{slot_id}/pause
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名，仅多模型加载时需要指定

请求体示例：

```
{
    "model": "model_alias"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "success": true,
    "id_slot": 0,
    "state": "paused"
}
```

错误响应（HTTP 400）：

错误信息	原因
`Slot is already paused`	该 slot 已经处于暂停状态
`Slot is not in a pausable state`	slot 处于 IDLE 或其他不可暂停的状态
`Invalid slot id: N`	slot ID 超出范围

POST `/slots/{slot_id}/resume`：恢复 slot

恢复已暂停 slot 的推理。

请求

```
POST /slots/{slot_id}/resume
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名，仅多模型加载时需要指定

请求体示例：

```
{
    "model": "model_alias"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "success": true,
    "id_slot": 0,
    "state": "resumed"
}
```

错误响应（HTTP 400）：

错误信息	原因
`Slot is not paused, cannot resume`	slot 当前不在暂停状态
`Invalid slot id: N`	slot ID 超出范围

4.3.3 示例

```
# 暂停 slot 0
curl -X POST http://localhost:8080/slots/0/pause \
    -H "Content-Type: application/json" \
    -d '{}'

# 恢复 slot 0
curl -X POST http://localhost:8080/slots/0/resume \
    -H "Content-Type: application/json" \
    -d '{}'
```

4.4 打断

4.4.1 功能介绍

Interrupt 允许直接终止指定 slot 的推理并将 slot 重置为 IDLE 状态。与 Pause 不同，Interrupt 会直接终止推理并释放 slot，可通过 `save_kvcache_path` 参数在打断后自动保存 KV Cache（一步完成打断+保存，无需单独调用 save 接口）。结合 KV Cache Save/Load 能力，可实现优先级打断等高级任务调度场景。

优先级打断的典型流程：

1. **启动低优先级任务**：在 slot 0 上以流式模式启动一个长文本推理任务
2. **打断低优先级任务并保存**：调用 `/slots/0/interrupt` 并指定 `save_kvcache_path`，一步完成打断 + KV Cache 保存（状态文件路径自动从 KV cache 路径派生：`xxx.kv` -> `xxx.kv.json`）
3. **断开连接**：断开低优先级任务的客户端连接，服务器检测到连接关闭后自动清理残留连接
4. **执行高优先级任务**：在 slot 0 上执行高优先级任务，等待其完成
5. **恢复低优先级任务**：重新发送与步骤 1 完全相同的 prompt，并在 `extra_body` 中指定 `load_kvcache_path`，服务器将自动加载 KV cache 和状态（从 cache 路径派生），从打断位置继续推理

4.4.2 API

POST `/slots/{slot_id}/interrupt`：打断 slot

打断指定 slot 的推理，将 slot 重置为 IDLE 状态。

请求

```
POST /slots/{slot_id}/interrupt
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名，仅多模型加载时需要指定
`save_kvcache_path`	string	否	指定后会在打断成功后自动保存 KV Cache 和推理状态（状态文件路径自动派生：`xxx.kv` -> `xxx.kv.json`）

请求体示例：

```
{
    "model": "model_alias",
    "save_kvcache_path": "/path/to/kvcache.kv"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "success": true,
    "id_slot": 0,
    "state": "idle",
    "kvcache_saved": true,
    "kvcache_path": "/path/to/kvcache.kv"
}
```

如果未指定 `save_kvcache_path`，则响应中不包含 `kvcache_saved` 和 `kvcache_path` 字段。

错误响应（HTTP 400 / 500）：

错误信息	原因
`Invalid slot id: N`	slot ID 超出范围
`Failed to interrupt slot`	打断失败（slot 不在推理状态）

4.4.3 示例

curl 示例

```
# 打断 slot 0
curl -X POST http://localhost:8080/slots/0/interrupt \
    -H "Content-Type: application/json" \
    -d '{}'

# 打断 slot 0 并自动保存 KV Cache
curl -X POST http://localhost:8080/slots/0/interrupt \
    -H "Content-Type: application/json" \
    -d '{"save_kvcache_path": "/data/kvcache/task.kv"}'
```

Python 示例（优先级打断完整流程）

```python
import requests
import json

BASE_URL = "http://127.0.0.1:8080"
SLOT_ID = 0

# 步骤 1：启动低优先级任务（长文本，流式模式）
# 使用子进程或独立线程发送请求，以便在收到若干 token 后打断

# 步骤 2：打断 slot 0 并自动保存 KV Cache
# 一步完成打断 + 保存，状态文件路径自动从 KV cache 路径派生：
#   /data/kvcache/task_low_priority.kv -> /data/kvcache/task_low_priority.kv.json
resp = requests.post(
    f"{BASE_URL}/slots/{SLOT_ID}/interrupt",
    json={
        "model": "",
        "save_kvcache_path": "/data/kvcache/task_low_priority.kv"
    },
    timeout=60
)
print(resp.json())  # {"success": true, "id_slot": 0, "state": "idle",
"kvcache_saved": true, "kvcache_path": "/data/kvcache/task_low_priority.kv"}

# 步骤 3：断开低优先级任务的客户端连接（kill 子进程 / 关闭 socket）
# 服务器检测到 is_connection_closed() → cancel_tasks() → 清理残留连接

# 步骤 4：执行高优先级任务
resp = requests.post(
    f"{BASE_URL}/v1/chat/completions",
    json={
        "model": "default",
        "messages": [{"role": "user", "content": "请用中文写一首关于春天的五言绝句。"}],
        "max_tokens": 256,
        "stream": False,
        "id_slot": SLOT_ID
    },
    timeout=120
)
print(resp.json()["choices"][0]["message"]["content"])

# 步骤 5：恢复低优先级任务
# 使用与步骤 1 完全相同的 prompt，并通过 extra_body 指定加载路径
# 状态文件自动从 KV cache 路径派生：
#   /data/kvcache/task_low_priority.kv -> /data/kvcache/task_low_priority.kv.json
resp = requests.post(
    f"{BASE_URL}/v1/chat/completions",
    json={
        "model": "default",
        "messages": [{"role": "user", "content": "请用中文详细写一篇关于人工智能发展历史的
综述文章..."}],
        "max_tokens": 4096,
        "stream": True,
        "id_slot": SLOT_ID,
        "extra_body": {
            "load_kvcache_path": "/data/kvcache/task_low_priority.kv"
        }
    },
    stream=True,
    timeout=300
)
```

```python
# 读取流式输出，将从停止位置继续生成
for line in resp.iter_lines(decode_unicode=True):
    if line and line.startswith("data: "):
        data_str = line[6:]
        if data_str == "[DONE]":
            break
        data = json.loads(data_str)
        content = data["choices"][0]["delta"].get("content", "")
        if content:
            print(content, end="", flush=True)
```

使用场景

- **多任务调度**：当有高优先级请求到达时，打断当前低优先级任务，执行高优先级任务后恢复
- **用户中断**：用户在等待长回复时发送新问题，打断旧任务，处理新问题后再恢复
- **资源管理**：动态分配 NPU 算力，确保关键任务优先完成

4.5 并发调度机制

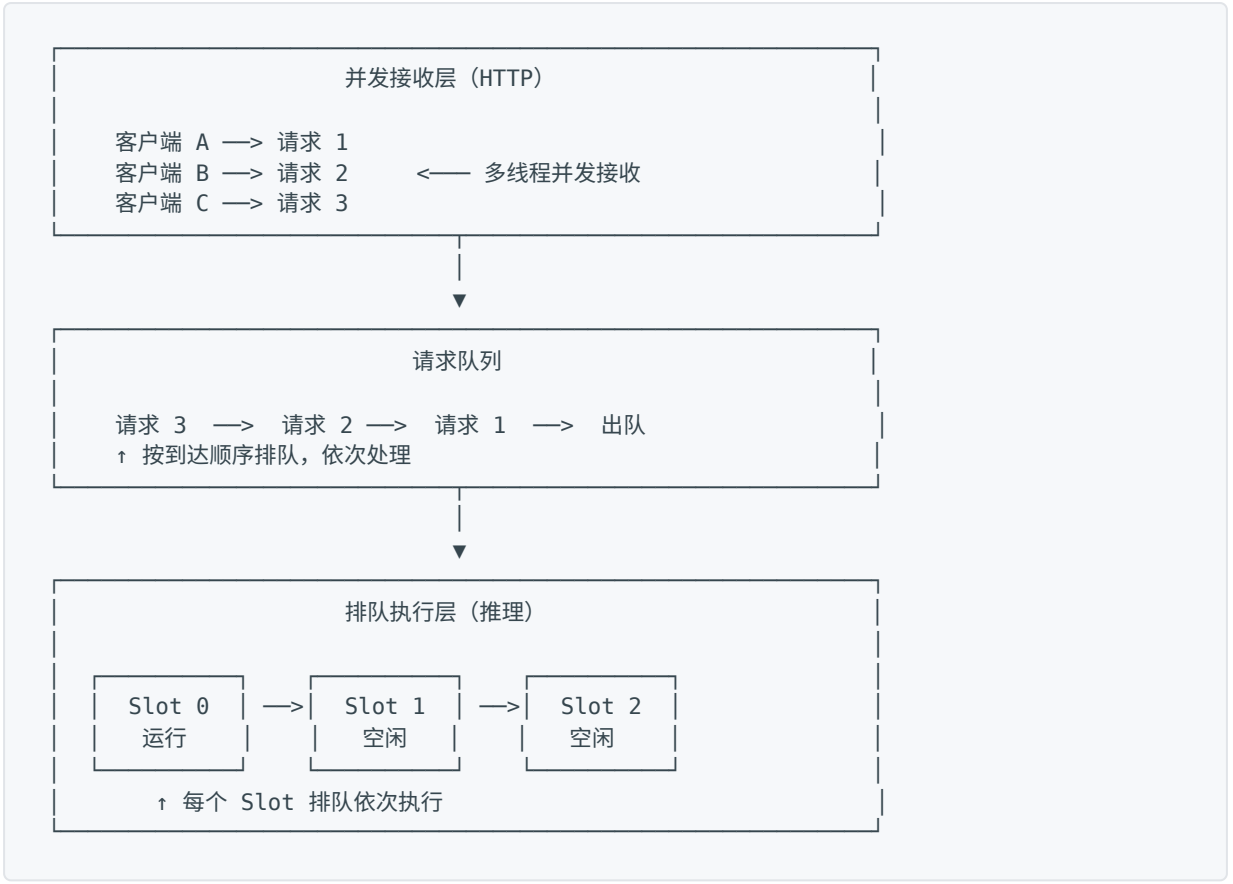
4.5.1 功能介绍

当 `--n-session` 设置为大于 1 时，RKLLM3 Server 会创建多个 Slot（会话槽位），每个 Slot 拥有独立的 KV Cache 和对话上下文。不同 Slot 之间可以实现对话隔离，服务端也可以同时接纳多个客户端请求。

4.5.2 并发调度模型

RKLLM3 Server 的并发调度模型可以概括为一句话：**HTTP 接收是并发的，但推理执行是排队串行的**。

具体来说，整个处理流程分为两层：



- **并发接收层**：HTTP 服务器可以同时接收多个客户端的请求，新请求到达时不会被正在执行的推理任务阻塞。
- **排队执行层**：所有推理请求按到达顺序排队，依次处理。每个请求分配到空闲 Slot 后执行推理，处理完成后释放 Slot 给下一个请求。

这种设计保证了服务端的稳定可靠：即使多个客户端同时发送请求，推理过程也不会因资源竞争而出错。

4.5.3 Slot 分配规则

- **未指定 Slot**：系统自动选择最久未使用的空闲 Slot，确保各 Slot 负载均衡
- **指定 Slot**（通过 `id_slot` 参数）：优先使用指定的 Slot；若该 Slot 正忙，请求进入排队等待
- 出现错误的 Slot 会被跳过，不再参与分配
- 当所有 Slot 都在忙碌时，新到达的请求会自动进入队列排队等待，不会立即返回错误。一旦某个 Slot 完成推理并释放，队列中的下一个请求会获得 Slot 并开始处理。

> **注意**：排队请求没有超时机制。如果请求长时间得不到处理，建议客户端设置合理的超时时间。

4.5.4 暂停/恢复/打断的响应

Pause（暂停）、Resume（恢复）、Interrupt（打断）操作会绕过请求队列直接执行，确保控制指令即时响应。相关接口详见 4.3 和 4.4 节。

---

5. KV Cache

5.1 保存和加载

5.1.1 功能介绍

KV Cache 保存/加载允许将指定 slot 的 KV Cache 保存到文件，或从文件加载恢复，实现对话 KV Cache 持久化。适用于以下场景：

- **对话续接**：先保存当前对话的 KV Cache，下次启动服务器后加载，无需重新输入历史消息即可续接对话
- **预热加速**：预先构造提示词生成 KV Cache 文件，避免每次冷启动时重复计算前缀

> **注意**：`save_kvcache_path` 和 `load_kvcache_path` 是每个请求的 `extra_body` 参数，两者不能在同一请求中同时使用。保存操作在本次生成结束后执行，加载操作在本次生成开始前执行。加载 KV Cache 后，请求中的 `messages` 仍会被处理（可用于追加新消息）。

5.1.2 API

通过 extra_body 参数操作

在 `/v1/chat/completions` 请求的 `extra_body` 中设置以下参数：

参数	类型	描述
`save_kvcache_path`	string	保存 KV Cache 到文件中（与 `load_kvcache_path` 不能同时使用）
`load_kvcache_path`	string	加载 KV Cache 文件（与 `save_kvcache_path` 不能同时使用）

POST `/slots/{slot_id}/kvcache/save`：保存指定 slot 的 KV Cache

将指定 slot 的 KV Cache 保存到文件。

> **注意**：此接口要求目标 slot 处于空闲状态（IDLE），即该 slot 没有正在进行的推理任务。如果 slot 正在处理中，接口将返回错误。

请求

```
POST /slots/{slot_id}/kvcache/save
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名或路径，仅多模型加载时需要指定
`path`	string	是	保存 KV Cache 的文件路径，请确保目标路径的父目录已存在

请求体示例：

```
{
    "model": "model_alias",
    "path": "/data/kvcache/backup.kv"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "success": true,
    "id_slot": 0,
    "path": "/data/kvcache/backup.kv"
}
```

错误响应（HTTP 400 / 500）：

错误信息	原因
`'path' is required for kvcache save`	请求中缺少 `path` 字段
`Invalid slot id: N`	slot ID 超出范围
`Failed to save kvcache for slot`	保存失败（可能 slot 正在处理中，或路径无效）

POST `/slots/{slot_id}/kvcache/load`：加载 KV Cache 到指定 slot

从文件加载 KV Cache 到指定 slot。

> **注意**：加载操作会覆盖目标 slot 当前的 KV Cache。加载后 slot 将继承文件中的 KV Cache 状态，可用于跨 slot 迁移或恢复之前的对话上下文。此接口要求目标 slot 处于空闲状态（IDLE）。

请求

```
POST /slots/{slot_id}/kvcache/load
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名或路径，仅多模型加载时需要指定
`path`	string	是	加载 KV Cache 的文件路径，请确保文件已存在

请求体示例：

```
{
    "model": "model_alias",
    "path": "/data/kvcache/backup.kv"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "success": true,
    "id_slot": 1,
    "path": "/data/kvcache/backup.kv"
}
```

错误响应（HTTP 400 / 500）：

错误信息	原因
`'path' is required for kvcache load`	请求中缺少 `path` 字段
`Invalid slot id: N`	slot ID 超出范围
`Failed to load kvcache for slot`	加载失败（可能 slot 正在处理中，或文件不存在）

5.1.3 使用限制

错误说明

错误信息	原因
`'save_kvcache_path' and 'load_kvcache_path' cannot both be set in the same request`	同一请求中同时设置了两个参数
`directory for 'save_kvcache_path' does not exist: ...`	保存路径的父目录不存在
`file for 'load_kvcache_path' does not exist: ...`	加载的文件不存在

5.1.4 示例

保存 KV Cache（Python）

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# 对话并保存 KV Cache
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "你好，我叫小明，请记住我的名字"}
    ],
    extra_body={"save_kvcache_path": "/data/kvcache/user_xiaoming.kv"}
)
print(completion.choices[0].message.content)
```

加载 KV Cache（Python）

```
# 加载 KV Cache 后续接对话
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "user", "content": "我叫什么名字? "}
    ],
    extra_body={"load_kvcache_path": "/data/kvcache/user_xiaoming.kv"}
)
print(completion.choices[0].message.content)
# 输出: 你叫小明
```

结合多 Session 使用

```
# Slot 0: 保存小明的 KV Cache
client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "你好，我叫小明"}],
    extra_body={"id_slot": 0, "save_kvcache_path": "/data/kvcache/slot0.kv"}
)

# Slot 1: 保存小红的 KV Cache
client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "你好，我叫小红"}],
    extra_body={"id_slot": 1, "save_kvcache_path": "/data/kvcache/slot1.kv"}
)

# 后续分别加载，各自独立 KV Cache
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "我叫什么名字? "}],
    extra_body={"id_slot": 0, "load_kvcache_path": "/data/kvcache/slot0.kv"}
)
print(completion.choices[0].message.content)  # 输出: 你叫小明
```

独立 API 示例（curl）

```
# 先在 slot 0 上完成一次对话（通过 chat/completions）
# 然后将 slot 0 的 KV Cache 保存到文件
curl -X POST http://localhost:8080/slots/0/kvcache/save \
    -H "Content-Type: application/json" \
    -d '{"path": "/data/kvcache/slot0_backup.kv"}'

# 将保存的 KV Cache 加载到 slot 1（跨 slot 迁移）
curl -X POST http://localhost:8080/slots/1/kvcache/load \
    -H "Content-Type: application/json" \
    -d '{"path": "/data/kvcache/slot0_backup.kv"}'

# 加载后，在 slot 1 上续接对话
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "messages": [{"role": "user", "content": "我叫什么名字? "}],
        "id_slot": 1
    }'
```

典型使用场景

- **对话备份与恢复**：在对话关键节点保存 KV Cache，服务器重启后加载恢复
- **跨 Slot 上下文迁移**：将 slot A 的对话上下文迁移到 slot B，释放 slot A 用于新任务
- **预计算加速**：提前构造提示词在某个 slot 上推理并保存 KV Cache，后续在其他 slot 上加载复用

5.2 Checkpoint

5.2.1 功能介绍

KV Cache 检查点功能允许在推理过程中保存 KV Cache 快照（适用于线性注意力或滑动窗口注意力模型，如 Qwen3.5、Gemma4 系列），后续可恢复快照以继续生成。主要用途：

- **多轮对话**：在关键对话节点保存 KV Cache，节省 prefill 时间
- **多 Session 场景**：每个并发 slot 可拥有独立的检查点策略
- **每次请求定制**：每次 API 请求可动态调整检查点参数

> **注意**：检查点参数仅对线性注意力或滑动窗口注意力模型生效（如 Qwen3.5、Gemma4 系列），对其他模型无影响。`checkpoint_start_pos` 和 `checkpoint_interval` 的值会被 NPU 运行时自动对齐到 128。`max_checkpoint_count` 若超过 `(ctx_size - checkpoint_start_pos) / checkpoint_interval`，将被自动 clamp 到此上限。

5.2.2 API

检查点参数可以在三个层级设置，优先级从高到低：

1. **每次请求**（最高优先级）：在 `/v1/chat/completions` 请求的 `extra_body` 中设置。如果请求中未指定这些检查点参数，slot 已有的 checkpoint 设置将被保留，不会重置为服务端默认值
2. **服务端 CLI**：`--checkpoint-start-pos`、`--checkpoint-interval`、`--max-checkpoint-count`、`--checkpoint-tail-overwrite`（使用 `#` 分隔符设置每个 slot 的值）
3. **服务端 params.json**：`"checkpoint-start-pos"`、`"checkpoint-interval"`、`"max-checkpoint-count"`、`"checkpoint-tail-overwrite"`（使用 JSON 数组设置每个 slot 的值）

参数说明

参数	CLI	JSON（`params.json`）	每次请求（`extra_body`）	说明
`checkpoint_start_pos`	`--checkpoint-start-pos N` 或 `N#N#...`	`"checkpoint-start-pos": N` 或 `[N, N, ...]`	`"checkpoint_start_pos": N`	开始保存检查点的 token 位置（≥ 0）；会自动对齐到 128
`checkpoint_interval`	`--checkpoint-interval N` 或 `N#N#...`	`"checkpoint-interval": N` 或 `[N, N, ...]`	`"checkpoint_interval": N`	保存检查点的间隔 token 数（0 = 禁用）；会自动对齐到 128
`max_checkpoint_count`	`--max-checkpoint-count N` 或 `N#N#...`	`"max-checkpoint-count": N` 或 `[N, N, ...]`	`"max_checkpoint_count": N`	从 start_pos 开始最多保存的检查点数量；应 ≤ `(ctx_size - start_pos) / interval`；超出时自动调整

参数	CLI	JSON（`params.json`）	每次请求（`extra_body`）	说明
`checkpoint_tail_overwrite`	`--checkpoint-tail-overwrite 0|1` 或 `0|1#...`	`"checkpoint-tail-overwrite": 0|1` 或 `[0, 1, ...]`	`"checkpoint_tail_overwrite": 0|1`	尾部覆盖模式。若为 `1`，则最后一个检查点将作为循环覆盖位，其他检查点保持固定位置不变（默认：`0`）

以上四个参数默认值均为 `0`，即默认不启用检查点保存。

5.2.3 示例

单 Session 示例

启动服务时设置检查点参数：

```
./rkllm3-server -m Qwen3.5-4B.rknn --vocab Qwen3.5-4B.tokenizer.gguf --embed
Qwen3.5-4B.embed.bin \
    --host 0.0.0.0 --port 8080 -c -1 --n-predict 2048 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 --max-checkpoint-count 4 --
checkpoint-tail-overwrite 1
```

或使用 `params.json`：

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "log-level": 1,
    "models": [{
        "model": "./Qwen3.5-4B.rknn",
        "vocab": "./Qwen3.5-4B.tokenizer.gguf",
        "embed": "./Qwen3.5-4B.embed.bin",
        "ctx-size": -1,
        "predict": 2048,
        "checkpoint-start-pos": 128,
        "checkpoint-interval": 128,
        "max-checkpoint-count": 4,
        "checkpoint-tail-overwrite": 1
    }]
}
```

每次 API 请求可覆盖检查点参数：

```
curl http://localhost:8080/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "messages": [{"role": "user", "content": "你好! "}],
        "extra_body": {
            "checkpoint_start_pos": 1024,
            "checkpoint_interval": 512,
            "max_checkpoint_count": 4,
            "checkpoint_tail_overwrite": 1
        }
    }'
```

多 Session + 各 Session 独立 Checkpoint 配置

CLI 示例（3 个 slot，各自不同的检查点配置）：

```
./rkllm3-server -m Qwen3.5-4B.rknn --vocab Qwen3.5-4B.tokenizer.gguf --embed
Qwen3.5-4B.embed.bin \
    --host 0.0.0.0 --port 8080 -c -1 --n-session 3 --n-predict 2048 \
    --checkpoint-start-pos 128#256#0 --checkpoint-interval 128#256#0 --max-
checkpoint-count 2#3#0 --checkpoint-tail-overwrite 1#0#0
```

params.json 示例（相同配置，使用 JSON 数组）：

```
{
    "host": "0.0.0.0",
    "port": 8080,
    "models": [{
        "model": "./Qwen3.5-4B.rknn",
        "vocab": "./Qwen3.5-4B.tokenizer.gguf",
        "embed": "./Qwen3.5-4B.embed.bin",
        "ctx-size": -1,
        "predict": 2048,
        "n-session": 3,
        "checkpoint-start-pos": [128, 256, 0],
        "checkpoint-interval": [128, 256, 0],
        "max-checkpoint-count": [8, 4, 0],
        "checkpoint-tail-overwrite": [1, 0, 0]
    }]
}
```

`"n-session": 3` 时，各 slot 的分配如下：

Slot	start_pos	interval	max_count	tail_overwrite	效果
0	128	128	8	1	高频检查点，尾部循环覆盖
1	256	256	4	0	中等频率检查点
2	0	0	0	0	禁用检查点

如果数组元素个数少于 `n-session`，剩余 slot 将使用标量默认值（或 `0`）。也支持标量值（如 `"checkpoint-interval": 128`），向后兼容地为所有 slot 设置相同的值。

6. 模型扩展

6.1 LoRA

6.1.1 功能介绍

RKLLM3 Server 支持通过 LoRA（Low-Rank Adaptation）对模型进行轻量级微调。单个 LoRA 权重文件可以包含多个适配器。您也可以重复传入 `--lora-weight` 参数（或在 JSON 配置中使用 `lora-weights` 数组）来加载多个独立的 `.lora_weight` 文件。所有从这些文件加载的适配器在所有会话（slot）之间共享，但每个会话可以在运行时独立配置各适配器的缩放比例。

CLI 方式（重复传入 `--lora-weight`）：

```
rkllm3-server -m Qwen2.5-3B-Instruct.rknn \
    --lora-weight adapter_a.lora_weight \
    --lora-weight adapter_b.lora_weight
```

JSON 配置方式（`lora-weights` 数组）：

```
{
    "models": [{
        "model": "Qwen2.5-3B-Instruct.rknn",
        "lora-weights": [
            "adapter_a.lora_weight",
            "adapter_b.lora_weight"
        ]
    }]
}
```

> 说明：`lora-weights` 与旧的单文件 `lora-weight` 字段相互兼容。若只加载一个文件，两者任选其一即可；同时配置时以 `lora-weights` 为准。

默认情况下，所有适配器的缩放比例（scale）将设置为 0。

6.1.2 API

GET `/lora-adapters`：获取所有 LoRA 适配器列表

此端点返回已加载的 LoRA 适配器列表。在启动服务器时通过 `--lora-weight` 参数添加适配器，例如：`--lora-weight Qwen2.5-3B-Instruct.lora_weight`。可重复传入多个 `--lora-weight` 以加载多个独立的 `.lora_weight` 文件。

请求

```
GET /lora-adapters
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名，仅多模型加载时需要指定
`id_slot`	int	否	指定要查询哪个会话的 LoRA 配置。当 `--n-session` 大于 1 时可用，默认值为 `0`

请求体示例：

```
{
    "model": "Qwen2.5-3B-Instruct",
    "id_slot": 0
}
```

返回结果

```
[
    {"name": "lora_slot0", "scale": 0.0},
    {"name": "lora_slot1", "scale": 0.0}
]
```

POST `/lora-adapters`：设置 LoRA 适配器列表

此端点用于为指定会话（slot）设置 LoRA 适配器的 scale。请注意，该 scale 会被每个请求中的 `lora` 字段值覆盖。

若要禁用某个适配器，可以将其从列表中移除，或者将其 scale 设置为 0。

请求

```
POST /lora-adapters
```

请求参数

参数	类型	必填	描述
`model`	string	否	模型别名，仅多模型加载时需要指定
`id_slot`	int	否	指定要更新哪个会话的 LoRA 配置。当 `--n-session` 大于 1 时可用，默认值为 `0`
`lora`	array	是	LoRA 适配器列表，每个对象包含 `name` 和 `scale` 字段

请求体示例：

```
{
    "model": "Qwen2.5-3B-Instruct",
    "id_slot": 0,
    "lora": [
        {"name": "lora_slot0", "scale": 0.2},
        {"name": "lora_slot1", "scale": 0.8}
    ]
}
```

所有会话共享同一组从 `--lora-weight` 加载的适配器，但每个会话可以拥有各自的 scale 值。

返回结果

成功响应（HTTP 200）：

```
{
    "success": true
}
```

错误响应（HTTP 400）：

错误信息	原因
`'lora' must be provided`	请求体中缺少 `lora` 字段
`'lora' must be an array of objects with 'name' and 'scale' fields`	`lora` 字段格式不正确
`Error: invalid adapter name`	指定了不存在的适配器名称
`Error: invalid id_slot`	指定了无效的 slot ID

每次请求覆盖

除了通过 `/lora-adapters` 端点全局设置外，还可以在每次推理请求中通过 `extra_body` 的 `lora` 字段按请求覆盖 LoRA 缩放比例：

```
{
    "messages": [{"role": "user", "content": "你好! "}],
    "lora": [
        {"name": "lora_slot0", "scale": 0.5},
        {"name": "lora_slot1", "scale": 1.0}
    ]
}
```

> **注意**：`extra_body` 中的 `lora` 字段会覆盖该 slot 已通过 `/lora-adapters` 设置的 scale 值。

6.2 Tie Word Embedding

6.2.1 功能介绍

Tie Word Embedding 是一种模型优化技术，将输入 embedding 权重与输出层共享，从而无需单独的 embed 文件（`--embed`），节省磁盘空间和内存。

> **注意**：此功能要求 RKNN 模型在转换时已开启 tie-word 支持。标准模型仍需提供 `--embed` 文件。
>
> 如果模型初始化时未启用 `--tie-word-embedding`，则每次请求的 `tie_word_embedding` 覆盖将被忽略（强制设为 0），服务端会输出警告日志。

6.2.2 API

CLI 参数

参数	类型	默认值	说明
`--tie-word-embedding`	`[0|1]`	`0`	启用 tie word embedding 模式，模型必须支持此功能才能启用。设为 `1` 时无需 `--embed` 文件

每次请求覆盖

若初始化时启用了 tie word embedding 模式，并且还提供了 embed 文件，则可以通过 `extra_body` 在每个 API 请求中覆盖 tie-word 设置：

```
{
    "messages": [{"role": "user", "content": "你好! "}],
    "tie_word_embedding": 0
}
```

若请求中未指定 `tie_word_embedding`，则使用服务端默认值（来自 CLI 或 `params.json`）。

6.2.3 示例

启动服务

```
# 标准模型（需要 embed 文件）
./rkllm3-server -m qwen2.5-3b.rknn --vocab qwen2.5-3b.tokenizer.gguf --embed
qwen2.5-3b.embed.bin --host 0.0.0.0 --port 8080

# Tie-word 模型（不需要 embed 文件）
./rkllm3-server -m Qwen3-VL-2B-llm-tie-word.rknn --weight Qwen3-VL-2B-llm-tie-
word.weight \
    --vocab Qwen3-VL-2B-llm_quant.tokenizer.gguf --tie-word-embedding 1 --host
0.0.0.0 --port 8080 -c 4096
```

Python 每次请求覆盖

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# 为本次请求覆盖 tie_word_embedding
completion = client.chat.completions.create(
    model="Qwen3-VL-2B",
    messages=[{"role": "user", "content": "你好! "}],
    extra_body={"tie_word_embedding": 0}
)
print(completion.choices[0].message.content)
```

6.3 Function Calling

6.3.1 功能介绍

RKLLM3 Server 支持 OpenAI 兼容的 Function Calling（工具调用），允许模型在推理过程中识别并调用外部工具。Jinja 模板默认开启，模型通过 chat template 解析请求中的 tools 定义，并在生成过程中输出结构化的 tool_calls。

> **注意**：只有当客户端请求中携带 `tools` 参数（且 `tool_choice` 不为 `"none"`）时，服务端才会设置 `parse_tool_calls = true` 来解析工具调用输出。如果客户端不传 `tools`，模型仍可能输出 `<tool_call>` 标签，但不会被解析为 OpenAI 兼容的 JSON `tool_calls` 字段。

6.3.2 API

Function Calling 通过 `/v1/chat/completions` 端点使用，在请求中添加以下参数：

请求参数

参数	类型	必填	描述
`tools`	array	否	工具定义列表，格式与 OpenAI 兼容
`tool_choice`	string/object	否	控制工具调用行为。可选值：`"auto"`（默认）、`"none"`、`"required"`，或指定具体工具
`parallel_tool_calls`	bool	否	是否允许并行调用多个工具（默认：`true`）

约束说明

- 使用 `tools` 时，服务端会自动设置 `parse_tool_calls = true` 来解析工具调用输出
- 当请求中包含 `tools` 时，服务端会自动重新生成 system prompt（合并 tools 定义），可在 `chat_template_kwargs` 中传入 `enable_thinking` 控制思考模式

请求体示例

```
{
    "model": "Qwen2.5-3B",
    "messages": [
        {"role": "user", "content": "北京今天天气怎么样？"}
    ],
    "tools": [
        {
            "type": "function",
            "function": {
                "name": "get_weather",
                "description": "获取指定城市的天气信息",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "city": {
                            "type": "string",
                            "description": "城市名称"
                        }
                    },
                    "required": ["city"]
                }
            }
        }
    ],
    "tool_choice": "auto"
}
```

返回结果

当模型决定调用工具时，响应中 `message` 包含 `tool_calls` 字段：

```
{
    "id": "chatcmpl-xxx",
    "object": "chat.completion",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": null,
                "tool_calls": [
                    {
                        "id": "call_xxx",
                        "type": "function",
                        "function": {
                            "name": "get_weather",
                            "arguments": "{\"city\": \"北京\"}"
                        }
                    }
                ]
            },
            "finish_reason": "tool_calls"
        }
    ]
}
```

6.3.3 示例

Python 调用

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "获取指定城市的天气信息",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "城市名称"}
                },
                "required": ["city"]
            }
        }
    }
]

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "北京今天天气怎么样？"}],
    tools=tools,
    tool_choice="auto"
)

msg = completion.choices[0].message
if msg.tool_calls:
    for tc in msg.tool_calls:
        print(f"Tool: {tc.function.name}, Args: {tc.function.arguments}")
else:
    print(msg.content)
```

多轮工具调用

```python
# 第一轮：模型返回 tool_calls
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "北京今天天气怎么样？"}],
    tools=tools
)

# 将 tool_calls 回传给模型
messages = [
    {"role": "user", "content": "北京今天天气怎么样？"},
    completion.choices[0].message.model_dump(),  # assistant 消息含 tool_calls
    {
        "role": "tool",
        "tool_call_id": "call_xxx",
        "content": '{"temperature": "25°C", "weather": "晴"}'
    }
]

# 第二轮：模型根据工具返回结果生成最终回复
completion2 = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=messages,
    tools=tools
)
print(completion2.choices[0].message.content)
```

6.4 Embedding 模型

6.4.1 功能介绍

RKLLM3 Server 支持词嵌入（Embedding）模型，通过 `--embedding` 参数启用。启动后提供 OpenAI 兼容的 `/v1/embeddings` 端点，用于将文本转换为向量表示，适用于语义搜索、文本分类、聚类等场景。

> **注意**：启用 `--embedding` 后，`/v1/completions` 端点将不可用。需要同时使用 Embedding 和 Chat 功能时，请启动多个服务实例。

6.4.2 API

POST `/v1/embeddings`：OpenAI 兼容的词嵌入 API

详见 OpenAI Embeddings API documentation。

请求

```
POST /v1/embeddings
```

请求参数

参数	类型	必填	描述
`model`	string	是	模型名称或别名
`input`	string/array	是	要嵌入的文本。可以是单个字符串或字符串数组。不能为空

参数	类型	必填	描述
`encoding_format`	string	否	返回格式。可选 `"float"`（默认）或 `"base64"`
`embd_normalize`	int	否	归一化方式。`2`（默认，L2 归一化），其他值按模型 pool 类型决定

请求体示例：

```
{
    "model": "Qwen3-Embedding-4B",
    "input": ["The food was delicious", "The service was great"],
    "encoding_format": "float"
}
```

返回结果

成功响应（HTTP 200）：

```
{
    "object": "list",
    "data": [
        {
            "object": "embedding",
            "index": 0,
            "embedding": [0.123, -0.456, 0.789, ...]
        },
        {
            "object": "embedding",
            "index": 1,
            "embedding": [-0.234, 0.567, -0.890, ...]
        }
    ],
    "model": "Qwen3-Embedding-4B",
    "usage": {
        "prompt_tokens": 10,
        "total_tokens": 10
    }
}
```

若 `encoding_format` 为 `"base64"`，`embedding` 字段为 base64 编码的 float32 字节数组。

错误响应

错误信息	原因
`This server does not support embeddings. Start it with \`--embedding``	服务未启用 embedding 模式
`"input" or "content" must be provided`	请求中缺少 `input` 字段
`"content" field is not OAI compatible`	使用了不被支持的 `content` 字段
`Input content cannot be empty`	输入文本为空
`The format to return the embeddings in. Can be either float or base64`	`encoding_format` 值不合法

错误信息	原因
`Pooling type 'none' is not OAI compatible`	模型池化类型不支持 embedding

6.4.3 示例

Python 调用

```python
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="sk-no-key-required"
)

# 单文本嵌入
response = client.embeddings.create(
    model="Qwen3-Embedding-4B",
    input="The food was delicious and the waiter...",
    encoding_format="float"
)
print(response.data[0].embedding)
```

7. 多模态输入

7.1 图像输入

7.1.1 功能介绍

RKLLM3 Server 支持多模态模型的图像输入。通过 `content` 数组中的 `image_url` 类型传入 base64 编码的图像数据。

7.1.2 API

在 `/v1/chat/completions` 请求的 `messages` 中使用 `image_url` 类型的 content：

```
{
    "type": "image_url",
    "image_url": {
        "url": "<base64_image>"
    }
}
```

7.1.3 示例

参见 3.2.2 多模态对话示例。

7.2 音频输入

7.2.1 功能介绍

RKLLM3 Server 支持多模态模型的音频输入。通过 `content` 数组中的 `input_audio` 类型传入 base64 编码的音频数据。

7.2.2 API

在 `/v1/chat/completions` 请求的 `messages` 中使用 `input_audio` 类型的 content：

```
{
    "type": "input_audio",
    "input_audio": {
        "data": "<base64_audio>",
        "format": "wav"
    }
}
```

7.2.3 示例

参见 3.2.2 多模态对话示例。

7.3 视频输入

7.3.1 功能介绍

对于视频理解任务，RKLLM3 Server 支持 qwen3_vl deepstack 模式的视频输入。使用 `video_frames` 内容类型，通过 `frames` 数组传入 base64 编码的视频帧。服务端会按帧对进行处理，使用 deepstack 视觉管线。

7.3.2 API

在 `/v1/chat/completions` 请求的 `messages` 中使用 `video_frames` 类型的 content：

```
{
    "type": "video_frames",
    "video_frames": {
        "frames": [
            "<base64_frame_1>",
            "<base64_frame_2>"
        ]
    }
}
```

7.3.3 示例

启动服务

```
./rkllm3-server -m Qwen3-VL-2B-llm.rknn --model2 Qwen3-VL-2B-vision.rknn --vocab
Qwen3-VL-2B-llm.tokenizer.gguf --embed Qwen3-VL-2B-llm.embed.bin --host 0.0.0.0 --
port 8080 -c 768 --n-predict 512 --repeat-penalty 1.1 --presence-penalty 1.0 --
frequency-penalty 1.0 --top-k 1 --top-p 0.8 --temp 0.8 --img-start "
<|vision_start|>" --img-end "<|vision_end|>" --img-content "<|image_pad|>" --img-
width 384 --img-height 384 --video-start "<|vision_start|>" --video-end "
<|vision_end|>" --video-content "<|video_pad|>"
```

Python 调用示例

```python
import base64
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:8080/v1",
    api_key="sk-no-key-required"
)

def encode_base64(file_path):
    with open(file_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

# 编码视频帧（例如按固定帧率从视频中提取）
frame_paths = ["frame_0001.jpg", "frame_0002.jpg", "frame_0003.jpg",
"frame_0004.jpg"]
base64_frames = [encode_base64(fp) for fp in frame_paths]

completion = client.chat.completions.create(
    model="Qwen3-VL-2B",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "video_frames",
                    "video_frames": {
                        "frames": [
                            f"{b64}"
                            for b64 in base64_frames
                        ]
                    },
                },
                {"type": "text", "text": "请描述一下视频中的内容?"},
            ],
        }
    ],
    stream=True,
    extra_body={
        "n_predict": 512,
        "id_slot": 0,
    }
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
    if chunk.choices[0].finish_reason == "stop":
        break
print('')
```

7.4 多模态请求格式

7.4.1 请求结构

多模态请求的 `messages` 中，`content` 字段为数组格式，支持混合多种内容类型。

7.4.2 Content 类型

类型	说明	示例
`text`	文本内容	`{"type": "text", "text": "请描述一下图片?"}`
`image_url`	图片输入（base64）	`{"type": "image_url", "image_url": {"url": "..."}}`
`input_audio`	音频输入（base64）	`{"type": "input_audio", "input_audio": {"data": "...", "format": "wav"}}`
`video_frames`	视频帧输入（base64 数组）	`{"type": "video_frames", "video_frames": {"frames": ["..."]}}`

7.4.3 示例

参见 3.2.2 多模态对话示例。

8. 多卡级联推理

8.1 功能介绍

多卡级联推理用于在 **多个 RK182X协处理器 上联合运行一个已被导出为K个分段（stage）的模型**。模型预先被拆分为多个分段文件（`xxx_seg0.rknn`、`xxx_seg1.rknn`、...），每一段加载到一张 RK182X协处理器 上，按 **stage0 → stage1 → ... → stageK-1** 的顺序流水线式推理，适用于单张卡内存不足以容纳完整模型、需要多张卡联合推理的场景。

> **提示**：分段模型的导出方法可参考 rknn3-model-zoo 中的多卡示例。

- **流水线并行**：前一段（stage i）的输出隐藏状态作为下一段（stage i+1）的输入，逐级传递，最后一层输出 logits。通过流水线并行（pipeline parallelism）实现多卡协同推理，同时提高prefill性能。
- **分桶处理**：prefill 按 `bucket-size`（默认 128）切分为多个 token 桶逐桶处理。
- **KV Cache**：多卡模式下同样支持 KV Cache 的保存与加载（自动按段拆分）。

8.2 配置参数

多卡级联推理主要配置参数如下，其余参数（如 `--vocab`、`--embed`、`--ctx-size`、`--predict`、采样参数、`--n-session` 等）与单卡模式一致，不再重复说明。

CLI 参数

参数	说明
`--stage-count N`	分段数量（>1 时启用多卡级联推理）
`-m, --model FNAME`	第一段模型路径，命名规范为 `xxx_seg0.rknn`
`--weight FNAME`	第一段权重路径（可不设置），多卡下命名规范为 `xxx_seg0.weight`
`--device-id STRING`	各段设备 ID，用 `#` 分隔（如 `0001:11:00.0#0003:31:00.0`）
`--bucket-size N`	多卡推理下每桶处理的 token 数（默认 128）
`--rope-tensor FNAME`	外置 rope cache 的 safetensors 文件路径（如 Qwen3.5、Gemma4 系列模型）

分段命名自动推导规则

只需指定第一段（`_seg0`）的模型/权重路径，其余段路径将根据`--stage-count`参数自动按 `_segN` 后缀推导：

- `model_seg0.rknn` → `model_seg1.rknn`、`model_seg2.rknn`、...
- 若首段文件名不含 `_segN`，会自动在 `.rknn` 扩展名前插入 `_segN`（如 `model.rknn` → `model_seg1.rknn`），并输出警告。
- 若使用 `dev:` 前缀指定设备侧路径（详见 2.3.5 节），前缀会在分段推导中自动保留（如 `dev:/data/model_seg0.rknn` → `dev:/data/model_seg1.rknn`）。
- `--device-id`可不设置，内部将会自动选择；若指定`--device-id`，则设备 ID 数量必须 ≥ `--stage-count`。

JSON 配置（`params.json`）

多卡推理模型的 `params.json` 基本格式（以双卡级联推理为例）：

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "timeout": 30,
  "models": {
    "Qwen3.5-9B": {
      "alias": "Qwen3.5-9B",
      "stage-count": 2,
      "model": "Qwen3.5-9B-llm_seg0.rknn",
      "weight": "Qwen3.5-9B-llm_seg0.weight",
      "vocab": "Qwen3.5-9B-llm.tokenizer.gguf",
      "embed": "Qwen3.5-9B-llm.embed.bin",
      "device-id": ["0001:11:00.0", "0003:31:00.0"],
      "bucket-size": 128,
      "ctx-size": 4096,
      "predict": 512,
      "n-session": 1,
      "checkpoint-start-pos": 128,
      "checkpoint-interval": 128,
      "max-checkpoint-count": 4,
      "checkpoint-tail-overwrite": 1,
      "rope-tensor": "Qwen3.5-9B-llm_seg0.safetensors"
    }
  }
}
```

> **注意：** `rope-tensor` 用于指定外置 rope cache 的 safetensors 文件。Qwen3.5 系列模型仅在采用外置 rope 时需要配置；Gemma-4 系列模型则必须配置。使用 rknn3-model-zoo 导出分段模型时会生成多个该文件，使用其中任意一个即可。

Gemma-4-12B-it 的配置示例如下：

```
{
  "host": "127.0.0.1",
  "port": 8080,
  "timeout": 30,
  "models": {
    "gemma-4-12b": {
      "alias": "gemma-4-12b",
      "stage-count": 2,
      "model": "gemma-4-12b-it_seg0.rknn",
      "weight": "gemma-4-12b-it_seg0.weight",
      "vocab": "gemma-4-12b-it.tokenizer.gguf",
      "embed": "gemma-4-12b-it.embed.bin",
      "device-id": ["0001:11:00.0", "0003:31:00.0"],
      "bucket-size": 128,
      "ctx-size": 4096,
      "predict": 512,
      "n-session": 1,
      "checkpoint-start-pos": 128,
      "checkpoint-interval": 128,
      "max-checkpoint-count": 4,
      "checkpoint-tail-overwrite": 1,
      "rope-tensor": "gemma-4-12b-it_seg0.safetensors"
    }
  }
}
```

8.3 启动示例

命令行启动（以双卡级联推理为例）

Qwen3.5-9B（2 段）：

```
./rkllm3-server --stage-count 2 --model Qwen3.5-9B-llm_seg0.rknn \
    --weight Qwen3.5-9B-llm_seg0.weight \
    --vocab Qwen3.5-9B-llm.tokenizer.gguf --embed Qwen3.5-9B-llm.embed.bin \
    --host 127.0.0.1 --port 8080 -c 4096 --n-predict 512 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 \
    --max-checkpoint-count 4 --checkpoint-tail-overwrite 1 \
    --rope-tensor Qwen3.5-9B-llm_seg0.safetensors
```

Gemma-4-12B-it（2 段）：

```
./rkllm3-server --stage-count 2 --model gemma-4-12b-it_seg0.rknn \
    --weight gemma-4-12b-it_seg0.weight \
    --vocab gemma-4-12b-it.tokenizer.gguf --embed gemma-4-12b-it.embed.bin \
    --host 127.0.0.1 --port 8080 -c 4096 --n-predict 512 \
    --checkpoint-start-pos 128 --checkpoint-interval 128 \
    --max-checkpoint-count 4 --checkpoint-tail-overwrite 1 \
    --rope-tensor gemma-4-12b-it_seg0.safetensors
```

配置文件启动

使用 `--params-file` 指定上述 `params.json`：

```
./rkllm3-server --params-file params.json
```

> **注意**：使用 `--params-file` 参数时，将忽略命令行的其余参数，仅 `params.json` 内的参数有效。
>
> **建议**：对于 Qwen3.5、Gemma4 系列模型，多轮对话场景下建议设置 **Checkpoint**，以便在后续对话中复用之前保存的 KV Cache 快照，节省 prefill 时间。多卡模式下 Checkpoint 策略会应用到所有分段会话。具体参数说明 5.2 Checkpoint。

8.4 KV Cache 保存与加载（多卡）

多卡模式下，KV Cache 的保存/加载会自动按段拆分，为每段生成独立的 KV Cache 文件。给定用户路径 `path.kv`：

- 保存：`path.kv` → `path_seg0.kv`、`path_seg1.kv`、...（在 `.kv` 前插入 `_segN`）
- 加载：同样按 `path_seg0.kv`、`path_seg1.kv`、... 分别加载到各段会话

通过接口保存/加载

- `POST /slots/{slot_id}/kvcache/save`：保存指定 slot 的 KV Cache（自动拆分为各段文件）
- `POST /slots/{slot_id}/kvcache/load`：加载指定 slot 的 KV Cache（自动从各段文件加载）

示例：

```
# 保存 slot 0 的 KV Cache（生成 test_slot_seg0.kv、test_slot_seg1.kv、...）
curl -X POST http://localhost:8080/slots/0/kvcache/save \
  -H "Content-Type: application/json" \
  -d '{"path": "/data/test_slot.kv"}'

# 加载到 slot 0（从 test_slot_seg0.kv、test_slot_seg1.kv、... 恢复）
curl -X POST http://localhost:8080/slots/0/kvcache/load \
  -H "Content-Type: application/json" \
  -d '{"path": "/data/test_slot.kv"}'
```

通过请求 extra_body 参数

在 `/v1/chat/completions` 请求的 `extra_body` 中设置 `save_kvcache_path` 或 `load_kvcache_path`（两者不能同时使用），保存/加载时同样自动按段拆分。

更多 KV Cache 说明参见 5.1 保存和加载。

---

9. Grammar 语法约束

Grammar（语法约束 / 结构化输出）用于限制模型的输出格式，使模型只能生成符合指定语法的内容，常用于需要模型输出稳定格式的场景（如枚举值、JSON 对象、固定标签等）。

grammar 功能默认关闭：需通过 `--grammar-on` 参数、配置文件 `"grammar-on": true`，或启动时配置有效全局默认 grammar（`--grammar` / `--grammar-file` / `-j` / `-jf`，配置后自动开启，无需再指定 `--grammar-on`）来开启（见 9.4）；未开启时，本章所述的各类语法约束均不生效，同时语法约束引擎不初始化，以节省主机内存。

9.1 请求字段与优先级

在 `POST /v1/chat/completions` 请求体中使用以下字段：

字段	类型	说明
`grammar`	string	用户自定义 EBNF 语法
`json_schema`	object	JSON Schema 对象
`response_format`	object	OpenAI 兼容：`{"type":"json_object", ...}` 或 `{"type":"json_schema","json_schema":{"schema":{...}}}`
`structural_tag`	object	结构化标签序列（const_string / regex / json_schema 组合），用于约束带固定标签的复杂输出（如 XML 工具调用格式），详见 9.5
`thinking_grammar`	string	开启 thinking 时 `json_schema` 约束的生效方式：`auto`（默认，服务端自动包裹）/ `off` / `manual`，详见 9.7。对 `structural_tag` 不生效：`structural_tag` 始终按原样下发，由调用方自带思考前缀
`thinking_end`	string	可选。覆盖 thinking 结束标记（默认从模型 chat template 自动检测）
`thinking_separator`	string	可选。覆盖 thinking 与最终答案之间的分隔符（默认 `"\n\n"`）
`grammar-on`	bool	可选。按请求覆盖 grammar 功能开关（默认继承启动配置值；启动时若已配置有效全局默认 grammar，开关自动开启，见 9.4）。语法约束引擎仅在启动时初始化：服务端未开启时，请求级 `"grammar-on": true` 无法补建引擎，各约束字段仍被忽略；服务端已开启时，可按请求传 `"grammar-on": false` 临时关闭本请求的语法约束，详见 9.4

互斥规则：`json_schema` 与 `grammar` 字段不能同时使用，同时下发会返回错误 `Cannot use both json_schema and grammar`。

> 注：当同时下发多个约束字段时，生效优先级为 `json_schema` / `response_format`（JSON Schema）> `structural_tag` > `grammar`（EBNF），且均高于 CLI 启动时设置的默认 grammar。

使用说明：

- **grammar 约束的是模型输出过程**：模型只能输出符合约束的内容；若模型推理结果与约束冲突，输出会被强制修正为合法序列。
- **每一轮请求的 grammar 相互独立**：本请求的 grammar 在结束后被清理，不会影响下一轮。

- **首次开启有一次性成本**：服务端在模型加载时会完成 grammar 的初始化；每次请求 set grammar 会重新编译，频繁切换 grammar 会有额外开销，建议在需要时再使用。

9.2 EBNF 语法约束

9.2.1 功能介绍

通过 `grammar` 字段下发自定义 EBNF（扩展巴科斯范式）语法，直接约束模型的输出格式，适用于枚举值、固定格式字符串等场景。

- **EBNF 默认根规则名为 `root`**，语法字符串需以 `root ::= ...` 开头定义入口规则。
- **正则必须匹配整个输出**（从头到尾），例如 `[0-9]{4}-[0-9]{2}-[0-9]{2}` 表示完整日期。

9.2.2 API

请求参数

参数	类型	必填	描述
`grammar`	string	否	用户自定义 EBNF 语法字符串，需以 `root ::= ...` 开头

9.2.3 示例

请求级 EBNF 约束

要求模型只输出 `yes` / `no` / `maybe`：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "is it working?"}],
    "grammar": "root ::= \"yes\" | \"no\" | \"maybe\""
  }'
```

9.3 JSON Schema 结构化输出

9.3.1 功能介绍

通过 `json_schema` 字段或 OpenAI 兼容的 `response_format` 字段，以 JSON Schema 描述输出结构，服务端将其转换为语法约束，保证输出为合法的 JSON 对象。适用于信息提取、结构化问答等场景。

9.3.2 API

请求参数

参数	类型	必填	描述
`json_schema`	object	否	JSON Schema 对象

参数	类型	必填	描述
`response_format`	object	否	OpenAI 兼容写法：`{"type":"json_object", ...}` 或 `{"type":"json_schema","json_schema":{"schema":{...}}}`，最终等价于 `json_schema`

9.3.3 示例

`json_schema`

要求输出含 `name`(string) 与 `age`(0~150 整数) 的 JSON 对象：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "介绍你自己"}],
    "json_schema": {
      "type": "object",
      "properties": {
        "name": {"type": "string"},
        "age":  {"type": "integer", "minimum": 0, "maximum": 150}
      },
      "required": ["name", "age"]
    }
  }'
```

OpenAI `response_format`

方式 A：`json_object`（任意合法 JSON 对象）：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "提取信息"}],
    "response_format": {"type": "json_object"}
  }'
```

方式 B：`json_schema`（带具体 schema）：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "提取天气信息"}],
    "response_format": {
      "type": "json_schema",
      "json_schema": {
        "schema": {
          "type": "object",
          "properties": {
            "city":        {"type": "string"},
            "temperature": {"type": "number"}
          },
          "required": ["city", "temperature"]
        }
      }
    }
  }'
```

9.4 启用 grammar 功能与全局默认 grammar

9.4.1 功能介绍

grammar 功能**默认关闭**，开启方式分两类：

- **默认关闭时**：服务端不初始化语法约束引擎（节省约 10MB 主机内存），本章所述的 `grammar` / `json_schema` / `response_format` / `structural_tag` 等约束字段均被忽略，服务端在请求确实携带约束字段时记录警告日志。
- **显式开启**：启动时通过 CLI 参数 `--grammar-on`，或配置文件（`--params-file`）中模型对象的 `"grammar-on": true` 字段。
- **自动开启**：启动时若已配置**有效全局默认 grammar**（`--grammar` / `--grammar-file` / `-j` / `-jf` 任一），则 grammar 功能自动开启，无需再指定 `--grammar-on`（配置默认语法内容本身就是明确的使用意图，服务端会记录一条自动开启的日志）。
- **按请求覆盖**：请求体支持可选的 `grammar-on` 布尔字段（默认继承启动配置值），详见 9.1。注意语法约束引擎仅在启动时初始化：服务端未开启时，请求级 `"grammar-on": true` 无法补建引擎；服务端已开启时，可按请求传 `"grammar-on": false` 临时关闭本请求的语法约束。

除启用开关外，rkllm3-server 还支持在**启动时**通过 CLI 参数设置全局默认 grammar。当某个请求没有携带 `grammar` / `json_schema` / `response_format` 字段时，会自动回退到这些默认值。

9.4.2 API

启用开关

方式	参数	说明
CLI	`--grammar-on`	开启 grammar 功能（布尔开关，无需取值）
CLI	`--grammar` / `--grammar-file` / `-j` / `-jf`	配置全局默认 grammar 时**自动开启** grammar 功能，无需再指定 `--grammar-on`
配置文件（`--params-file`）	`"grammar-on": true`	写在对应模型对象内；布尔值，数字 `0` / `1` 也兼容，非法值忽略并记录警告

请求级覆盖

参数	类型	说明
`grammar-on`	bool	按请求覆盖开关（默认继承启动配置值）

全局默认 grammar 的 CLI 参数

CLI 参数	取值	说明
`--grammar GRAMMAR`	string	直接传入 EBNF 语法字符串
`--grammar-file FNAME`	文件路径	从文件读取 EBNF 语法
`-j, --json-schema SCHEMA`	JSON 字符串	传入 JSON Schema
`-jf, --json-schema-file FILE`	文件路径	从文件读取 JSON Schema

覆盖关系：请求级 `grammar` / `json_schema` / `response_format` > CLI 默认 grammar > 无约束。

> **注意**：配置了全局默认 grammar（`--grammar` / `--grammar-file` / `-j` / `-jf` 任一）时 grammar 功能**自动开启**；仅当启动时既未带 `--grammar-on` / `"grammar-on": true`、也未配置任何全局默认 grammar 时，功能才保持关闭。 **注意**：`-j` / `-jf` 依赖服务端内部做 JSON Schema 到语法格式的转换，对外部 `$ref` 的支持有限；带外部 `$ref` 的复杂 schema 请先用脚本预转换为 EBNF，再通过 `--grammar-file` 使用。

9.4.3 示例

示例一：开启 grammar 功能并固定输出 `yes` / `no`（`--grammar` 已配置默认语法，`--grammar-on` 可省略，此处保留以示显式）：

```
./rkllm3-server -m model.rknn --grammar-on --grammar 'root ::= "yes" | "no"'
```

示例二：开启 grammar 功能并加载 grammar 文件（`--grammar-file` 已配置默认语法，`--grammar-on` 可省略）：

```
./rkllm3-server -m model.rknn --grammar-file ./grammars/json.ebnf
```

示例三：启动时指定 JSON Schema（`-j` 已配置默认语法，`--grammar-on` 可省略）：

```
./rkllm3-server -m model.rknn -j '{"type":"object","properties":{"name":
{"type":"string"}},"required":["name"]}'
```

示例四：启动时加载 JSON Schema 文件（`-jf` 已配置默认语法，`--grammar-on` 可省略）：

```
./rkllm3-server -m model.rknn -jf ./schemas/person.json
```

示例五：通过配置文件开启（`params.json` 中在对应模型对象内添加 `"grammar-on"` 字段）：

```
{
    "host": "127.0.0.1",
    "port": 8080,
    "models": [
        {
            "model": "./Qwen3.5-4B.rknn",
            "vocab": "./Qwen3.5-4B.tokenizer.gguf",
            "grammar-on": true
        }
    ]
}
```

启动命令：`./rkllm3-server --params-file params.json`。注意使用 `--params-file` 时命令行其余参数被忽略，`--grammar-on` 与 `"grammar-on": true` 不要混用（以配置文件为准）。

示例六：服务端已开启时，按请求临时关闭本请求的语法约束：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "is it working?"}],
    "grammar-on": false,
    "grammar": "root ::= \"yes\" | \"no\" | \"maybe\""
  }'
```

> 说明：服务端带 `--grammar-on`（或已配置默认 grammar）启动时，该请求的 `"grammar-on": false` 使 `grammar` 字段被忽略，模型自由输出；启动时既未配置默认 grammar 也未带 `--grammar-on` 时，即便传 `"grammar-on": true`，`grammar` 字段同样被忽略（引擎无法在运行时补建）。

9.5 structural_tag 结构化约束

9.5.1 功能介绍

当输出需要"固定标签包裹 + 部分自由内容"的结构时（例如强制模型输出 `<tool_call><function=工具名>参数JSON</function></tool_call>` 这类带固定标签的工具调用格式），单独的 `grammar`（EBNF）或 `json_schema` 都难以直接表达。此时可以使用 `structural_tag` 字段：它以**序列**的方式描述输出结构，序列中每个元素可以是固定字符串、正则或 JSON Schema。

9.5.2 语法结构

`structural_tag` 是一个 JSON 对象，结构如下：

```
{
  "type": "structural_tag",
  "format": {
    "type": "sequence",
    "elements": [
      {"type": "const_string", "value": "<固定字符串>"},
      {"type": "regex", "pattern": "<正则表达式>"},
      {"type": "json_schema", "json_schema": { "<JSON Schema>" }, "style":
"qwen_xml"}
    ]
  }
}
```

序列元素类型：

元素类型	字段	说明
`const_string`	`value`	固定字符串，输出时原样出现
`regex`	`pattern`	正则表达式，约束该位置可匹配的字符序列
`json_schema`	`json_schema`	内嵌 JSON Schema，约束该位置的合法 JSON 对象；可选 `style`（如 `qwen_xml`）指定工具调用的参数风格

9.5.3 示例

强制模型以 Qwen XML 格式输出工具调用 `<tool_call><function=工具名>参数JSON</function></tool_call>`：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "请调用 ping 工具。"}],
    "tools": [
      {
        "type": "function",
        "function": {
          "name": "ping",
          "description": "Return a simple ping result.",
          "parameters": {"type": "object", "properties": {}}
        }
      }
    ],
    "tool_choice": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "const_string", "value": "\n<tool_call>\n<function="},
          {"type": "regex", "pattern": "[A-Za-z0-9_-]{1,64}"},
          {"type": "const_string", "value": ">\n"},
          {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}, "style": "qwen_xml"},
          {"type": "const_string", "value": "\n</function>\n</tool_call>"}
        ]
      }
    }
  }'
```

> 说明：- `structural_tag` 的优先级高于 `grammar`（EBNF），低于 `json_schema` / `response_format`。- 序列中的 `regex` 需能匹配对应位置的实际内容（如工具名），其余固定字符串按原样输出。- Qwen XML 原生协议的完整结构为 `<tool_call>\n<function=工具名>参数JSON</function>\n</tool_call>`：开标签 `<tool_call>` 不可省略，否则输出有闭合无开始，带 `tools` 时响应解析会失败。- 该功能适合工具调用、结构化标签等场景；普通 JSON 结构化输出优先使用 `json_schema` / `response_format` 即可。

9.6 结构化输出与工具调用模式组合

9.6.1 功能介绍

当请求同时携带 `tools`（且 `tool_choice` 不为 `"none"`）时，服务端会根据请求内容决定是否对工具调用的输出做语法约束。

9.6.2 组合规则

请求内容	grammar 行为
只传 `tools`，不传 grammar	不设置语法约束，模型自由输出工具调用，由服务端解析
`tools` + `grammar`（EBNF）	使用请求中的 EBNF 语法约束工具调用输出

请求内容	grammar 行为
`tools` + `json_schema` / `response_format`	使用请求中的 JSON Schema 约束工具调用输出
`tools` + `structural_tag`	使用请求中的结构化标签序列约束工具调用输出（推荐用于带固定标签的工具调用格式，见 9.5）
`tools` 且请求未携带任何 grammar	回退到 CLI 启动参数 `--grammar` / `--grammar-file`（EBNF），若启动参数也没有则不设约束

9.6.3 示例

工具调用时强制模型输出合法 JSON 对象（用请求中的 EBNF 语法）：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "查一下北京天气"}],
    "tools": [{"type":"function","function":{"name":"get_weather","parameters":
{"type":"object","properties":{"city":{"type":"string"}},"required":["city"]}}}],
    "tool_choice": "auto",
    "grammar": "root ::= object"
  }'
```

> 说明： - 请求中的 `grammar` 必须是 **EBNF** 格式的语法字符串。 - 更推荐在 tool call 场景下使用 `json_schema` / `response_format`，它们对 JSON 结构描述更直接，约束也更明确。 - 不使用语法约束时，工具调用的结构化输出完全依赖模型能力与服务端解析，存在解析失败的可能；如需保证格式稳定，建议配合语法约束使用。

9.7 结构化输出与思考模式组合

9.7.1 功能介绍

开启 thinking（思考模式）后，模型输出分为两部分：先输出思考过程，再输出最终答案。RKNN3 runtime 从第一个生成 token 开始应用 grammar，且不会自动检测 thinking，因此 thinking 开启时的 grammar 必须覆盖完整生成流：`思考文本 -> 思考结束标记 -> 分隔符 -> 最终答案`。

服务端只对 `json_schema` / `response_format` 提供自动处理：`thinking_grammar` 默认 `auto`，服务端自动把 schema 包裹为覆盖完整流的 Structural Tag，思考不受约束，最终答案符合 schema（详见示例一）。

对 `structural_tag`，服务端**始终按原样下发**，不做任何包裹或裁剪：完整流由调用方自己编写，开放全部约束逻辑，（详见示例二）。

支持范围（重要）

约束字段	与 thinking 组合
`json_schema`	支持：`thinking_grammar=auto`（默认）时服务端自动包裹完整流，见示例一
`response_format`（`json_object` / `json_schema` 类型）	支持，等价于 `json_schema`

约束字段	与 thinking 组合
`structural_tag`	支持：始终按原样下发，需自带思考前缀（完整自写，见示例二 / 示例三）
`grammar`（EBNF）	不支持

开启 thinking 时若下发 `grammar`（EBNF），服务端会记录警告日志，约束将作用于模型全部输出（含思考部分），可能导致思考内容与最终答案输出异常。如需在 thinking 场景下约束最终答案，请改用 `json_schema` 或 `structural_tag`。

9.7.2 `thinking_grammar` 字段

该字段仅影响 `json_schema` / `response_format`，对 `structural_tag` 不生效：

取值	行为
`auto`（默认）	服务端检测本次请求 thinking 是否实际开启；开启时自动把 `json_schema` 包裹为覆盖完整流的 Structural Tag，思考部分不受约束。**用户无需任何额外配置。**
`off`	不做调整，schema 约束作用于模型全部输出（含思考部分）。适合 thinking 已关闭、或明确希望约束整个输出的场景
`manual`	对 `json_schema` 同 `off`，由调用方自行处理完整流

`auto` 模式下，`json_schema` 的两个可选覆盖字段（一般无需设置）：

字段	默认值	说明
`thinking_end`	从模型 chat template 自动检测	思考结束标记，如 `</think>`
`thinking_separator`	`"\n\n"`	思考结束标记与最终答案之间的固定分隔

> 注：这两个字段只影响 `json_schema` 的自动包裹。`structural_tag` 按原样下发，思考结束标记与分隔符直接写在 tag 元素中（见示例二）。

9.7.3 示例

示例一：开启 thinking + `json_schema`（`auto` 自动包裹，流式）

用户只写熟悉的 `json_schema`，其余全部自动处理。thinking 场景建议配合流式输出，可实时看到思考过程：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "计算 2 + 2 等于多少? 先思考，然后给出答
案"}],
    "json_schema": {
      "type": "object",
      "properties": {"answer": {"type": "integer"}},
      "required": ["answer"]
    },
    "stream": true
  }'
```

SSE 响应中 delta 按顺序推送：

1. 思考阶段：`delta.reasoning_content` 逐步推送思考文本（自由生成，不受 schema 约束）；
2. 思考结束后：`delta.content` 推送最终答案，保证拼合后为符合 schema 的 JSON，如 `{"answer": 4}`；
3. 最后收到 `data: [DONE]`。

非流式（`"stream": false` 或不传）同样支持，行为等价：

- `reasoning_content`：模型的思考过程，为自由文本，不受 schema 约束；
- `content`：思考结束后的最终答案，保证为符合 schema 的 JSON。

服务端实际下发给推理引擎的完整约束为：

```
思考文本(任意) -> </think> -> "\n\n" -> 符合 schema 的 JSON
```

显式下发 `thinking_grammar` / `thinking_end` / `thinking_separator`

自动包裹使用的思考结束标记与分隔符默认从模型 chat template 自动检测（如 `</think>` 与 `"\n\n"`）。需要显式控制时，可在请求中直接下发这三个字段：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "计算 2 + 2 等于多少? 先思考，然后给出答
案"}],
    "json_schema": {
      "type": "object",
      "properties": {"answer": {"type": "integer"}},
      "required": ["answer"]
    },
    "thinking_grammar": "auto",
    "thinking_end": "</think>",
    "thinking_separator": "\n\n",
    "stream": true
  }'
```

字段说明：

- `thinking_grammar`：`auto`（默认，服务端自动包裹，本例显式写出，行为与不传相同）/ `off`（不包裹，schema 约束作用于全部输出）/ `manual`（同 `off`，完整流由调用方自行处理）；
- `thinking_end`：覆盖自动检测的思考结束标记，示例中显式写为 `</think>`；模板检测不准时，改为模型实际的思考结束标记；
- `thinking_separator`：覆盖思考结束标记与最终答案之间的分隔符（默认 `"\n\n"`），示例中显式写为 `"\n\n"`；如需改变思考与答案之间的衔接，可改为其他字符串（如 `"\n"`）。

> 注：这三个字段只影响 `json_schema` / `response_format` 的自动包裹；`structural_tag` 按原样下发，思考结束标记与分隔符直接写在 tag 元素中（见示例二）。

示例二：开启 thinking + 完整自写 `structural_tag`（开放全部约束逻辑）

调用方完全掌控约束逻辑：structural_tag 自带思考前缀（任意思考文本 + 思考结束标记 + 分隔符）+ 最终答案约束，服务端按原样下发，不做任何包裹。下面以 Qwen3.5 Native XML 工具调用协议为例：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "请调用 ping 工具。"}],
    "thinking_grammar": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "tag", "begin": "", "content": {"type": "any_text"}, "end": "
</think>"},
          {"type": "const_string", "value": "\n\n"},
          {"type": "const_string", "value": "\n<tool_call>\n<function="},
          {"type": "regex", "pattern": "ping"},
          {"type": "const_string", "value": ">\n"},
          {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}, "style": "qwen_xml"},
          {"type": "const_string", "value": "\n</function>\n</tool_call>"}
        ]
      }
    },
    "stream": false,
    "max_tokens": 512
  }'
```

完整约束流：

```
思考文本(任意) -> </think> -> "\n\n" ->
<tool_call>\n<function=ping>\n{...}\n</function>\n</tool_call>
```

要点：

- 前 2 个元素即完整思考流：思考文本不受限（`any_text`），以 `</think>` 结束，随后固定输出分隔符 `\n\n`；其余元素描述最终答案（Qwen3.5 Native XML 协议）。
- 思考结束标记与分隔符直接写在 tag 元素中；模型模板的标记不同时（可用 `thinking_end` 查询自动检测值），同步修改这两个元素。
- 思考过程照常返回在 `reasoning_content` 中，工具调用格式仅约束思考结束后的输出。
- JSON-in-XML / Hermes 协议（Qwen3 风格 `<tool_call>\n{"name":..., "arguments":...}\n</tool_call>`）同样支持：将思考前缀 2 个元素保留，最终答案部分换成 `triggered_tags` 节点，完整示例见示例三。

示例三：开启 thinking + 完整自写 `structural_tag`（JSON-in-XML / Hermes）

思考前缀 2 个元素包在外层 `sequence` 中，最终答案部分为 `triggered_tags` 节点，输出 JSON-in-XML 风格的工具调用：

```
curl http://127.0.0.1:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "messages": [{"role": "user", "content": "请调用 ping 工具。"}],
    "thinking_grammar": "auto",
    "structural_tag": {
      "type": "structural_tag",
      "format": {
        "type": "sequence",
        "elements": [
          {"type": "tag", "begin": "", "content": {"type": "any_text"}, "end": "
</think>"},
          {"type": "const_string", "value": "\n\n"},
          {
            "type": "triggered_tags",
            "triggers": ["<tool_call>\n"],
            "tags": [
              {
                "type": "tag",
                "begin": "<tool_call>\n{\"name\": \"ping\", \"arguments\": ",
                "content": {"type": "json_schema", "json_schema": {"type": "object",
"additionalProperties": true}},
                "end": "}\n</tool_call>"
              }
            ],
            "at_least_one": true,
            "stop_after_first": true
          }
        ]
      }
    },
    "stream": false,
    "max_tokens": 512
  }'
```

完整约束流：

```
思考文本(任意) -> </think> -> "\n\n" -> <tool_call>\n{"name": "ping", "arguments":
{...}}\n</tool_call>
```

要点：

- 前 2 个元素仍是完整思考流；`triggered_tags` 节点本身不能包含前置自由文本，因此思考前缀必须包在外层 `sequence` 中，不能写进节点内部。
- 固定的 name 前缀写在 tag 的 `begin` 中，`content` 只约束 arguments 对象；多个函数时为每个函数各加一条 tag，并保证 `triggers` 是对应 tag `begin` 的无歧义前缀。
- `at_least_one` + `stop_after_first`：思考结束后必须且只生成一个 tool_call。
- 请求未下发 `tools`：Qwen3.5 模板的原生协议为 Native XML（见示例二），与 JSON-in-XML 输出不一致，带 `tools` 时服务端响应解析会失败并返回 500（见注意事项 2）。不下发 `tools` 时，`reasoning_content` 为思考文本，原始工具调用直接返回在 `content` 中，由调用方自行解析：

```
<tool_call>
{"name": "ping", "arguments": {}}
</tool_call>
```

9.7.4 注意事项

1. **`max_tokens` 需预留思考空间**：thinking 开启时，思考文本同样消耗生成 token，`max_tokens` 过小会在思考阶段即被截断，导致最终答案不完整。建议在常规取值基础上适当放大（如 512 以上）。
2. **工具调用协议需与模型一致**：structural_tag 描述的工具调用协议决定模型输出格式，而服务端响应解析按模型 chat template 的原生协议进行。两者不一致时（例如在 Qwen3.5 上使用 JSON-in-XML 格式），最终答案会解析失败并返回 500。规避方式：(a) 不下发 `tools`，让原始输出直接返回在 `content` 中自行解析（完整示例见示例三）；(b) structural_tag 采用与模型原生协议一致的格式（Qwen3.5 用方案一 Native XML，Qwen3 用方案二 JSON-in-XML）。
3. **与 tools 组合**：thinking + `tools` + `json_schema` 同时下发时，按 9.1 优先级由 `json_schema` 生效，工具调用格式约束不再生效。thinking 下的工具调用格式约束请改用 `structural_tag`（见示例二）。
4. **structural_tag 按原样下发**：服务端不检测、不包裹、不修改 `structural_tag`。若忘记写思考前缀，约束将从第一个生成 token 开始生效，思考阶段会被最终答案约束压制，导致输出异常。
5. **模板未提供思考结束标记时**：`auto` 对 `json_schema` 的自动包裹不会生效，保持原始行为（约束从首 token 生效，不破坏现有请求）；`structural_tag` 不受影响，按原样下发。

附录

A. API 端点索引

Endpoint	方法	功能	所在章节
`/health`	GET	健康检查	2.4
`/v1/chat/completions`	POST	对话补全	3.1, 3.2, 3.3
`/v1/models`	GET	模型列表	B.1
`/v1/embeddings`	POST	词嵌入	6.4
`/lora-adapters`	GET	获取 LoRA 适配器列表	6.1
`/lora-adapters`	POST	设置 LoRA 适配器列表	6.1
`/slots/{slot_id}/pause`	POST	暂停 slot	4.3
`/slots/{slot_id}/resume`	POST	恢复 slot	4.3
`/slots/{slot_id}/interrupt`	POST	打断 slot	4.4
`/slots/{slot_id}/kvcache/save`	POST	保存 KV Cache	5.1
`/slots/{slot_id}/kvcache/load`	POST	加载 KV Cache	5.1

B. OpenAI 兼容说明

B1. POST `/v1/chat/completions`：聊天补全

给定 `messages` 中的 CHATML 形式的 JSON 描述，返回预测的补全。支持同步和流模式。只有具有聊天模板的模型才可以在此端点下正常使用。默认情况下，将使用 CHATML 模板。

详见 OpenAI Chat Completions API documentation。

`extra_body` 支持的选项（非 OpenAI 兼容）：

参数	类型	描述
`n_predict`	int	当前请求要预测的 token 数量
`chat_template_kwargs`	object	当前请求的对话模板参数，如 `{"enable_thinking": false}` 用于控制 thinking 输出
`id_slot`	int	指定当前 slot 的 id（当 `--n-session` 参数大于 1 时可用，默认为 0）
`save_kvcache_path`	string	保存 KV Cache 到文件中（与 `load_kvcache_path` 不能同时使用）
`load_kvcache_path`	string	加载 KV Cache 文件（与 `save_kvcache_path` 不能同时使用）
`top_k`	int	top-k 采样
`top_p`	float	top-p 采样
`temperature`	float	温度采样
`repeat_penalty`	float	惩罚重复 token 序列

参数	类型	描述
`frequency_penalty`	float	重复 alpha 频率惩罚
`presence_penalty`	float	重复 alpha 存在惩罚
`lora`	array	用于设定 LoRA 适配器列表。列表中的每个对象必须包含 `name` 和 `scale` 字段。如果某个 LoRA 适配器未在列表中指定，其 scale 将默认为 `0.0`
`tie_word_embedding`	int	为当前请求启用 tie word embedding 模式（0 或 1）。设为 1 时无需单独指定 embed 文件
`checkpoint_start_pos`	int	开始保存检查点的 token 位置（≥ 0），会自动对齐到 128
`checkpoint_interval`	int	保存检查点的间隔 token 数（0 = 禁用），会自动对齐到 128
`max_checkpoint_count`	int	最多保存的检查点数量
`checkpoint_tail_overwrite`	int	尾部覆盖模式（`0` / `1`）。若为 `1`，最后一个检查点将作为循环覆盖位

C. 错误码

RKLLM3 Server 的错误响应格式兼容 OpenAI API 规范。所有错误响应均为 JSON 格式，包含 `code`、`message` 和 `type` 三个字段。

C1. 错误类型速查表

错误类型	HTTP 状态码	`type` 字段	说明
`ERROR_TYPE_INVALID_REQUEST`	400	`invalid_request_error`	请求参数无效或格式错误
`ERROR_TYPE_AUTHENTICATION`	401	`authentication_error`	API Key 认证失败
`ERROR_TYPE_PERMISSION`	403	`permission_error`	无权限访问（保留，当前未使用）
`ERROR_TYPE_NOT_FOUND`	404	`not_found_error`	请求的资源不存在
`ERROR_TYPE_SERVER`	500	`server_error`	服务器内部错误
`ERROR_TYPE_NOT_SUPPORTED`	501	`not_supported_error`	请求的功能不支持（自定义）
`ERROR_TYPE_UNAVAILABLE`	503	`unavailable_error`	服务暂时不可用，如模型加载中（自定义）

C2. 常见错误消息及排查

HTTP 状态码	错误消息	可能原因	解决方案
400	`"input" or "content" must be provided`	请求体缺少 `input` 或 `content` 字段	检查 Embedding 请求体格式
400	`Input content cannot be empty`	输入内容为空	确保请求中包含有效文本
400	`Invalid slot id`	Slot ID 不存在	检查 Slot ID 是否正确
400	`Slot is already paused`	Slot 已处于暂停状态	仅在非暂停状态下调用暂停
400	`Slot is not paused, cannot resume`	Slot 未处于暂停状态	仅在暂停状态下调用恢复
400	`Slot is not in a pausable state`	Slot 处于 IDLE 或其他不可暂停的状态	仅在推理进行中时调用暂停
400	`'path' is required for kvcache save`	缺少 `path` 参数	传入有效的缓存路径
400	`'path' is required for kvcache load`	缺少 `path` 参数	传入有效的缓存路径
404	`File Not Found`	请求的静态文件不存在	检查文件路径是否正确
500	`Server context not available`	服务器未正确初始化	重启服务并检查模型加载
500	`Failed to pause/resume/interrupt slot`	Slot 操作失败	检查 Slot 状态后重试
500	`Failed to save/load kvcache for slot`	KV Cache 读写失败	检查磁盘空间和路径权限
501	`This server does not support completions`	以 embedding 模式启动	不带 `--embeddings` 重新启动
501	`This server does not support embeddings`	未启用 embedding 模式	带 `--embedding` 重新启动
503	`Loading model`	模型正在加载中	等待模型加载完成后再请求

D. 配置项汇总

D1. Server 配置

CLI参数	JSON (`params.json`)	默认值	描述
`--host`	`"host"`	`127.0.0.1`	监听 IP 地址
`--port`	`"port"`	`8080`	监听端口
`-to, --timeout`	`"timeout"`	`600`	读写超时时间（秒）
`--log-level`	`"log-level"`	`0`	日志输出等级（详见 2.5.1 节）
`--device-id`	`"device-id"`	-	多设备时指定设备 ID

D2. 模型配置

CLI参数	JSON (`params.json`)	默认值	描述
`-m, --model`	`"model"`	-	LLM 模型路径
`--weight`	`"weight"`	-	LLM 权重路径
`--model2`	`"model2"`	-	Vision 模型路径
`--weight2`	`"weight2"`	-	Vision 权重路径
`--model3`	`"model3"`	-	Audio 模型路径
`--weight3`	`"weight3"`	-	Audio 权重路径
`--vocab`	`"vocab"`	-	词汇表路径
`--embed`	`"embed"`	-	Embed 文件路径
`--embed-ple`	`"embed-ple"`	-	Gemma4 per_layer_inputs
`--rope-tensor`	`"rope-tensor"`	-	Gemma4 rope cache
`--mel-filter`	`"mel-filter"`	-	Audio mel filters
`--lora-weight`	`"lora-weight"`	-	LoRA 权重路径
`--decrypt-key, --key`	`"decrypt-key"`	-	LLM 加密模型密钥包路径
`--decrypt-key2, --key2`	`"decrypt-key2"`	-	Vision 加密模型密钥包路径
`--decrypt-key3, --key3`	`"decrypt-key3"`	-	Audio 加密模型密钥包路径
`-c, --ctx-size`	`"ctx-size"`	`4096`	上下文大小（支持 `#` 分隔符）
`-n, --predict`	`"predict"`	`-1`	预测 token 数（支持 `#` 分隔符）
`-a, --alias`	`"alias"`	-	模型别名
`--core-mask`	`"core-mask"`	自动	LLM NPU 核心掩码
`--core-mask2`	`"core-mask2"`	自动	Vision NPU 核心掩码
`--core-mask3`	`"core-mask3"`	自动	Audio NPU 核心掩码
`--img-width`	`"img-width"`	`0`	输入图像宽度
`--img-height`	`"img-height"`	`0`	输入图像高度
`--chat-template-file`	`"chat-template-file"`	-	自定义 Jinja 模板文件
`--embedding`	`"embedding"`	`false`	是否为词嵌入模型
`--tie-word-embedding`	`"tie-word-embedding"`	`0`	启用 tie word embedding
`--reasoning-format`	`"reasoning-format"`	`none`	推理格式（`none` / `deepseek` ）
`--reasoning`	`"reasoning"`	`auto`	推理开关（`on` / `off` / `auto` ）
`--record-path`	`"record-path"`	-	用户请求保存目录
`--n-session`	`"n-session"`	`1`	并发 session 数量
`--stage-count`	`"stage-count"`	`1`	多卡级联推理分段数量
`--bucket-size`	`"bucket-size"`	`128`	多卡级联推理prefill每桶 token 数